Inferenza di una rete neurale nel kernel Linux

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale e lo inoltra. Nel banco è il core 6.
XDP	Il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato.
Pipeline	Una delle versioni del programma che esegue la rete neurale (P1, P1.5, P2, P3).
baseline / rxonly	I due programmi di riferimento: la baseline inoltra senza rete neurale, rxonly riceve e basta.
Coda	I 256 posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Coda d'uscita	La coda del cavo virtuale verso il nodo successivo. Con più core sul nodo ce n'è una per core, svuotata da un core suo.
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
M/s	Milioni di pacchetti al secondo.
ns, µs, ms	Nanosecondi (miliardesimi di secondo), microsecondi (1 µs = 1 000 ns), millisecondi (1 ms = 1 000 µs).
Gbit/s	Miliardi di bit al secondo. Con pacchetti da 64 byte, 1 Gbit/s ≈ 1,95 milioni di pacchetti al secondo.
Costo per pacchetto	1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
Tempo di CPU per pacchetto	Quanto tempo il core del nodo lavora davvero per ogni pacchetto: occupazione del core × 1 diviso la capacità. Coincide con il costo per pacchetto quando il nodo lavora il 100% del tempo.
Scrittore	Chi mette pacchetti in una coda. Con una scheda di rete vera è la scheda stessa: uno per coda.

P-core, E-core, LP E-core	I tre tipi di core del processore: veloci (Redwood Cove), efficienti (Crestmont, in moduli da quattro che dividono cache L2 e frequenza) e a basso consumo (fuori dalla cache L3, al massimo 2,5 GHz).
Istruzioni per pacchetto	Quante istruzioni macchina il core del nodo esegue per ogni pacchetto, contate dal processore stesso (contatori hardware). Dipendono dal codice, non dal core.
Ciclo, IPC	Il ciclo è il battito dell'orologio del core (3,5 miliardi al secondo a 3,5 GHz). IPC = istruzioni eseguite per ciclo: dice quanto lavoro il core fa a ogni battito, e dipende dal core.
BPF_PROG_TEST_RUN	Una funzione del kernel che esegue il programma XDP su un pacchetto preparato, in un ciclo, senza traffico vero: misura il costo del solo programma.
xdp_gen	Il generatore di traffico del banco: manda al nodo pacchetti nella stessa forma in cui XDP li riceve da una scheda di rete.
T1, T2, T3	Tre letture dell'orologio in una versione strumentata del programma: all'ingresso (T1), prima di inoltrare (T2), all'arrivo al nodo successivo (T3). T2−T1 è il solo programma, T3−T2 il trasporto.
Minimo e media	Il minimo è il pacchetto più veloce di una finestra (tutto in cache); la media è il costo tipico di un pacchetto sotto traffico.
Popolazione	I 91 modelli sintetici su cui gira ogni misura: diversi per composizione dell'ingresso, profondità, larghezza, uscite, pesi e pesi a zero. Nessuna cifra è di un modello solo.
Mediana [min–max]	La mediana sulla popolazione e, fra parentesi, il valore più basso e il più alto: dal modello più leggero al più pesante.
MAC	Moltiplicazione e somma (multiply-accumulate): l'operazione base di uno strato. Le MAC eseguite contano una feature one-hot come una colonna sola.
Legge di costo	costo della rete = a + b × MAC eseguite + c × strati, stimata con i minimi quadrati su tutti i modelli densi. L'errore si misura su modelli lasciati fuori dalla stima.

Come si misura e da dove vengono i tempi

Il banco. Un solo portatile fa tutto: il generatore di traffico, il nodo che esegue la rete neurale e il nodo successivo che riceve i pacchetti inoltrati. Sono collegati da cavi virtuali (`veth`) con XDP nativo: il programma del nodo vede ogni pacchetto nel primo punto del kernel in cui arriva, come su una scheda di rete vera. Ogni ruolo ha un core suo (nodo sulla CPU 6, uscita sulla 8, generatore sulla 10).

Le condizioni della macchina (`host_conditions.py`). Lasciato a sé stesso, il portatile cambia frequenza fra 0,4 e 4,8 GHz, addormenta i core e mette processi qualunque sul core che misura. Prima di ogni misura il banco fissa la frequenza dei core del banco a 3,5 GHz, spegne gli stati di sonno profondi, sposta il resto del sistema su altri core e accende un monitor (frequenza misurata, temperatura, throttling, alimentazione). Tutto viene ripristinato a fine misura e scritto accanto ai numeri. Una misura in cui la macchina esce dalle condizioni dichiarate è marcata e non entra nelle conclusioni.

Il generatore (`xdp_gen`). Produce pacchetti già nella forma in cui XDP li riceve da una scheda di rete (frame XDP grezzi): niente struttura `skb`, niente copie. Il nodo li vede come li vedrebbe da un driver vero. Può spingere al massimo o tenere un ritmo fissato, con i pacchetti distanziati in modo uniforme.

La popolazione di modelli (`model_population.py`). Ogni misura gira su **91 modelli sintetici**, non su un modello solo. Coprono 7 composizioni dell'ingresso (solo colonne dense, solo one-hot, miste; da 5 a 117 ingressi), da 1 a 6 strati nascosti, da 2 a 8 neuroni per strato, da 3 a 7 uscite, da 21 a 1 349 pesi. Sono divisi in quattro famiglie:

- **base** (53): le combinazioni di composizione, profondità e larghezza;
- **replica** (20): sei forme con semi diversi, per vedere quanto il costo dipende dai pesi estratti;
- **sparsità** (8): quattro forme con il 50 e il 90 % dei pesi a zero;
- **imbuto** (10): cinque forme con lo stesso numero di pesi (~592) distribuito su 1–5 strati, su due composizioni.

I test più lunghi (traffico crescente, LP E-core, due core, classi, TTL misti, taglie) girano su sottoinsiemi di 6–14 modelli, scelti dal più leggero al più pesante.

Come si leggono le cifre. Nessuna cifra delle slide è di un modello solo. Ogni numero è una **mediana sulla popolazione**, un intervallo **[minimo–massimo]** dal modello più leggero al più pesante, il numero di modelli su cui un'affermazione regge («85 modelli su 85»), oppure una **legge** stimata su tutti i modelli: costo = a + b × moltiplicazioni eseguite + c × strati.

Cinque modi di misurare, ognuno per una domanda diversa:

1. **A massima spinta** (`bench_throughput --mode compare`): il generatore manda più pacchetti di quanti il nodo ne regga, la coda resta piena e il core del nodo lavora il 100% del tempo. Si misurano la **capacità** (pacchetti al secondo) e il **tempo di CPU per pacchetto**, più istruzioni, cicli e mancate di cache dai contatori del processore. Slide 3–8, 10, 12–15, 17, 19–23.
2. **A traffico crescente** (`bench_bitrate`): da 0,5 a 12 milioni di pacchetti al secondo; si contano i pacchetti inviati, quelli ricevuti dal programma e quelli inoltrati. Dice **dove** si perde oltre la capacità. Slide 9.
3. **Con tre orologi** (`bench_throughput --mode rates`): una versione del programma legge l'orologio all'ingresso (T1), subito prima di inoltrare (T2) e all'arrivo al nodo successivo (T3). **T2−T1 è il solo**

programma, T3–T2 il trasporto. Il traffico sta sotto la capacità (0,25 e 0,5 milioni di pacchetti al secondo). Slide 11 e 16.

4. **Il programma caricato** (`prog_census`): il kernel dice quanti programmi e quante istruzioni ha caricato ogni pipeline. È una proprietà del programma, non una misura di tempo. Slide 4, 17, 20, 23.
5. **Il programma da solo** (`bench_solo`, cioè `BPF_PROG_TEST_RUN`): il kernel esegue il programma su un pacchetto preparato, con gli stessi ingressi del traffico, senza ricevere né trasmettere. Serve per ciò che il traffico non misura: le letture di tabella e i salti fra programmi per pacchetto (con una build che li conta), e il tempo per installare un modello. Slide 17 e 18.

Da dove vengono i tempi (P-core a 3,5 GHz):

- **ricevere il frame**: circa 40 ns (rxonly, il programma che riceve e butta);
- **leggere l'intestazione, decrementare il TTL, inoltrare**: circa 27 ns in più. Con la ricezione fanno i ~67 ns della baseline, il costo fisso del nodo, uguale con qualunque modello (64–69 ns sui 91);
- **la rete neurale**: quello che ogni pipeline aggiunge sopra la baseline. Mediane: 52 ns per P1, 59 per P1.5, 225 per P2, 420 per P3; dal modello più leggero al più pesante 19–155 ns per P1, 129–446 per P2, 240–852 per P3;
- **dentro la rete neurale**: il costo segue una legge. Circa 0,26 ns per moltiplicazione con i pesi compilati (P1, P1.5), 0,42 per P2, 0,77 per P3, che legge ogni peso da una tabella; P3 paga in più circa 46 ns per ogni strato, P2 circa 20. Pesano le **letture di tabella** (in mediana 5 per pacchetto con P1, 12 con P2, 35 con P3, fino a 54 sui modelli più profondi) e i **salti fra programmi** (uno per P1, P1.5 e P2; per P3 uno per strato): sono il prezzo della configurabilità;
- **il trasporto verso il nodo successivo**: 187–221 ns (redirect, cavo virtuale, ricezione), uguale per tutte le pipeline e tutti i modelli. Pesa sulla latenza del pacchetto; nel tempo del nodo entra solo l'accodamento del redirect, la ricezione la paga il core d'uscita.

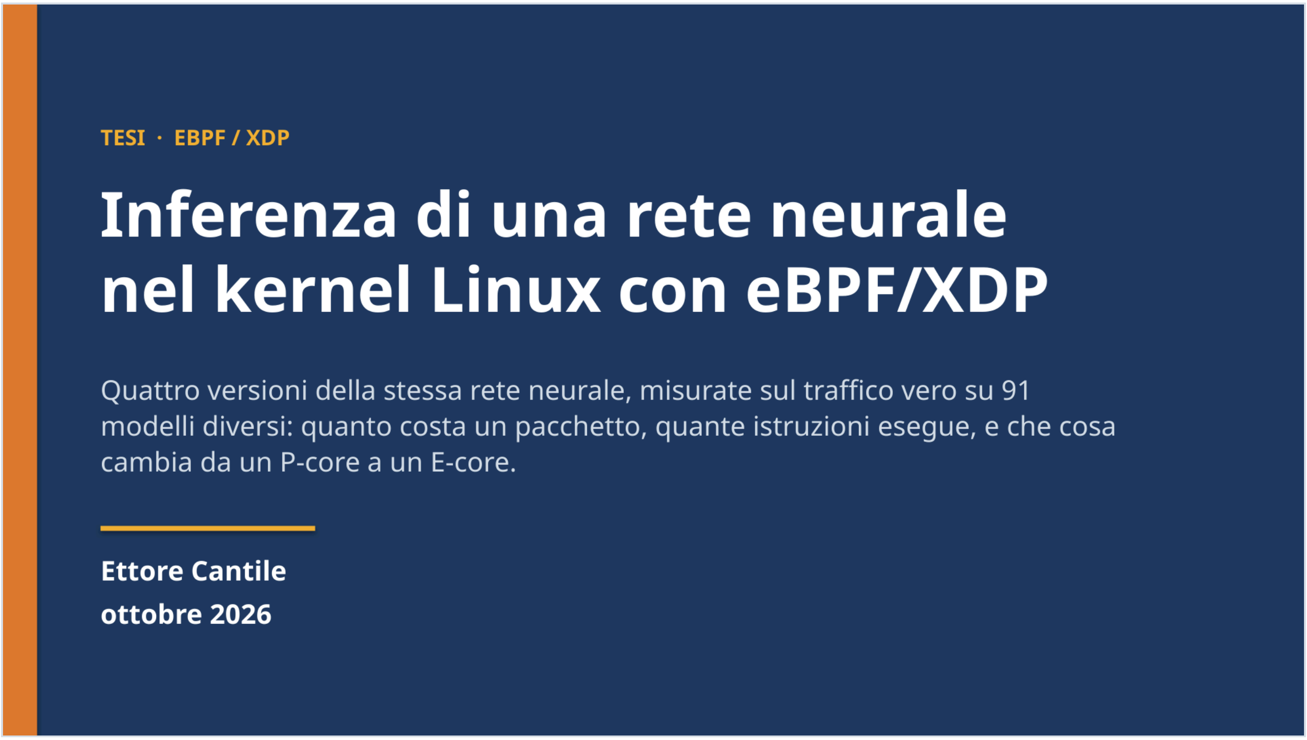
Esempio: due misure della stessa rete. Per P3 la mediana di T2–T1 a traffico leggero, meno la baseline, sta a 1,02 [0,94–1,11] volte il tempo di CPU a nodo saturo meno la baseline, sui 91 modelli; per P2 a 1,03 [0,94–1,09]. Due metodi indipendenti danno lo stesso costo della rete su ogni modello.

I limiti del banco. Il cavo virtuale non è una scheda di rete: non ci sono interruzioni hardware né trasferimenti dalla scheda alla memoria (DMA), e il generatore sta sulla stessa macchina. I costi del programma sono quelli veri; il costo di ricezione su una scheda vera può essere diverso. Le cifre assolute valgono a 3,5 GHz e vanno citate con il tipo di core. I modelli sono sintetici: coprono forme e composizioni, non l'accuratezza di un modello addestrato.

Quanto è pulito il dato. Su 435 misure una sola è fuori dalle conclusioni (macchina disturbata). Le curve a traffico crescente durano ~160 s e su questo portatile un evento di throttling di pacchetto arriva circa una volta al minuto: 8 curve sono tenute senza le finestre colpite (17 su 1 920), purché ogni punto conservi almeno 2 giri puliti su 3.

SLIDE 1

Copertina



TESI · EBPF / XDP

Inferenza di una rete neurale nel kernel Linux con eBPF/XDP

Quattro versioni della stessa rete neurale, misurate sul traffico vero su 91 modelli diversi: quanto costa un pacchetto, quante istruzioni esegue, e che cosa cambia da un P-core a un E-core.

Ettore Cantile
ottobre 2026

La tesi esegue una piccola rete neurale **dentro il kernel Linux**, nel punto in cui il pacchetto è appena arrivato (XDP), per decidere da quale porta farlo uscire. La stessa rete è scritta in quattro modi diversi, da completamente compilata a completamente configurabile, e la domanda è quanto costa, per ogni pacchetto, ciascuna scelta.

La risposta non deve dipendere da un modello particolare. Per questo ogni misura gira su una popolazione di **91 modelli**, diversi per composizione dell'ingresso, profondità, larghezza, uscite, pesi e pesi a zero, e ogni conclusione è un intervallo o una legge su tutti.

La presentazione ha due parti:

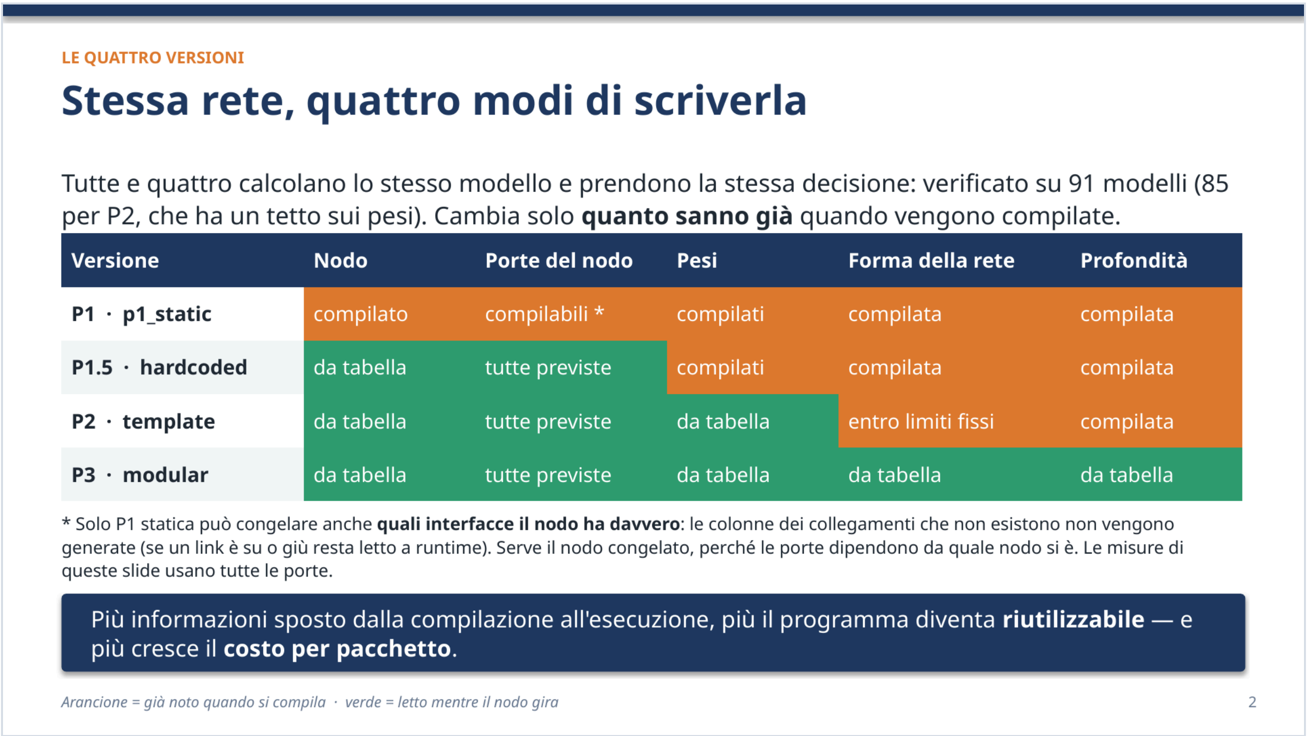
1. **Il nodo sotto traffico vero** (slide 3–16): un generatore manda pacchetti veri al nodo e si misura quanti ne regge, quante istruzioni esegue per ognuno, e che cosa cambia se il nodo gira su un core veloce (P-core), efficiente (E-core) o a basso consumo (LP E-core).
2. **La rete neurale sotto traffico** (slide 17–23): quanto costa la sola rete neurale al crescere della forma del modello e della rete, e la legge che permette di stimarlo prima di misurarlo; più il costo di cambiare modello (slide 18), che è un tempo del piano di controllo.

Le cifre vengono dalla campagna dell'11–12 ottobre 2026 sul portatile di laboratorio (Intel Core Ultra 7 155H, Ubuntu 24.04), con i core del banco a frequenza fissa: 3,5 GHz per P-core ed E-core, 2,5 GHz per il LP E-core.

Prima delle slide, la pagina «Come si misura e da dove vengono i tempi» descrive il banco, la popolazione, i modi di misurare e la scomposizione del costo di un pacchetto. Le spiegazioni delle slide la danno per letta.

SLIDE 2

Le quattro versioni



LE QUATTRO VERSIONI

Stessa rete, quattro modi di scriverla

Tutte e quattro calcolano lo stesso modello e prendono la stessa decisione: verificato su 91 modelli (85 per P2, che ha un tetto sui pesi). Cambia solo **quanto sanno già** quando vengono compilate.

Versione	Nodo	Porte del nodo	Pesi	Forma della rete	Profondità
P1 · p1_static	compilato	compilabili *	compilati	compilata	compilata
P1.5 · hardcoded	da tabella	tutte previste	compilati	compilata	compilata
P2 · template	da tabella	tutte previste	da tabella	entro limiti fissi	compilata
P3 · modular	da tabella	tutte previste	da tabella	da tabella	da tabella

* Solo P1 statica può congelare anche **quali interfacce il nodo ha davvero**: le colonne dei collegamenti che non esistono non vengono generate (se un link è su o giù resta letto a runtime). Serve il nodo congelato, perché le porte dipendono da quale nodo si è. Le misure di queste slide usano tutte le porte.

Più informazioni sposto dalla compilazione all'esecuzione, più il programma diventa **riutilizzabile** — e più cresce il **costo per pacchetto**.

Arancione = già noto quando si compila · verde = letto mentre il nodo gira

Tutte e quattro calcolano **lo stesso modello** e prendono **la stessa decisione** sullo stesso pacchetto. Lo verifica, su ogni modello della popolazione, il confronto esatto fra il riferimento intero e il programma eBPF (`BPF_PROG_TEST_RUN`, 200 ingressi per modello): P1 e P3 coincidono su 91 modelli su 91, P2 su 85 su 85. Gli altri 6 modelli superano il tetto sui pesi di P2, che le altre pipeline non hanno.

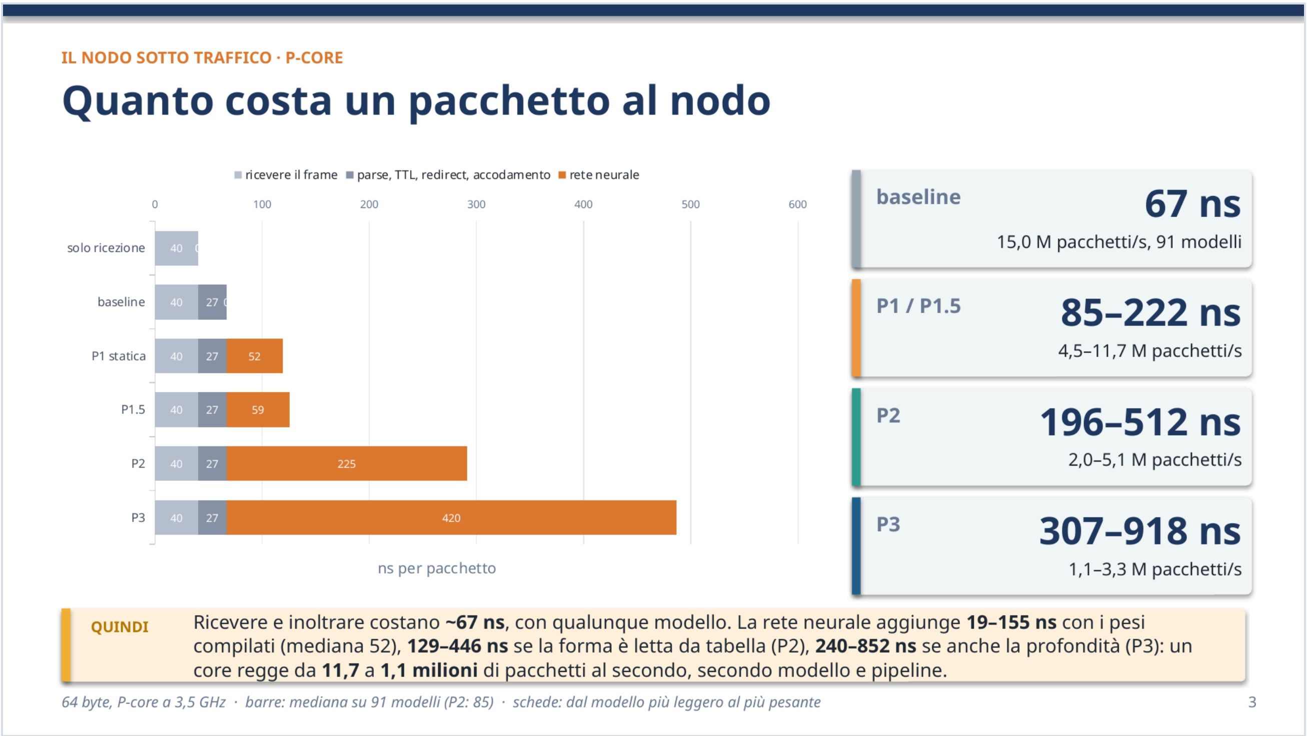
Cambia solo che cosa è già scritto nel programma quando lo si compila (arancione) e che cosa invece il programma legge da tabelle mentre gira (verde).

- **P1 statica**: pesi e identità del nodo scritti nel programma. Un binario per ogni nodo.
- **P1.5 hardcoded**: pesi nel programma, nodo letto da tabella. Un binario per modello.
- **P2 template**: anche la forma della rete si legge da tabella; nel programma ci sono solo i valori massimi ammessi. Un programma per molti modelli.
- **P3 modular**: anche il numero di strati si legge da tabella, e ogni strato è un programma a parte raggiunto con un salto. Un programma per tutto.

La regola che le slide successive misurano: più informazioni si spostano dalla compilazione all'esecuzione, più il programma diventa riutilizzabile, e più cresce il costo per pacchetto. I colori delle pipeline (P1 rosso, P1.5 arancione, P2 verde, P3 blu) restano gli stessi in tutti i grafici.

SLIDE 3

Quanto costa un pacchetto al nodo



IL NODO SOTTO TRAFFICO · P-CORE

Quanto costa un pacchetto al nodo

baseline **67 ns**
15,0 M pacchetti/s, 91 modelli

P1 / P1.5 **85–222 ns**
4,5–11,7 M pacchetti/s

P2 **196–512 ns**
2,0–5,1 M pacchetti/s

P3 **307–918 ns**
1,1–3,3 M pacchetti/s

QUINDI Ricevere e inoltrare costano **~67 ns**, con qualunque modello. La rete neurale aggiunge **19–155 ns** con i pesi compilati (mediana 52), **129–446 ns** se la forma è letta da tabella (P2), **240–852 ns** se anche la profondità (P3): un core regge da **11,7** a **1,1 milioni** di pacchetti al secondo, secondo modello e pipeline.

64 byte, P-core a 3,5 GHz · barre: mediana su 91 modelli (P2: 85) · schede: dal modello più leggero al più pesante

Come si misura. Un generatore (xdp_gen) manda al nodo più pacchetti di quanti ne possa elaborare. La coda davanti al nodo resta sempre piena, il core del nodo lavora il 100% del tempo, e il ritmo a cui elabora è la sua capacità. Il costo per pacchetto è 1 diviso la capacità, ed è anche il tempo di CPU per pacchetto.

Le barre scompongono il costo in tre pezzi, con la mediana sui 91 modelli:

- **ricevere il frame** (grigio chiaro): quanto costa rxonly, il programma che riceve e butta, circa 40 ns;
- **parse, TTL, redirect, accodamento**: quello che la baseline fa in più, circa 27 ns;
- **rete neurale** (arancione): quello che ogni pipeline aggiunge sopra la baseline.

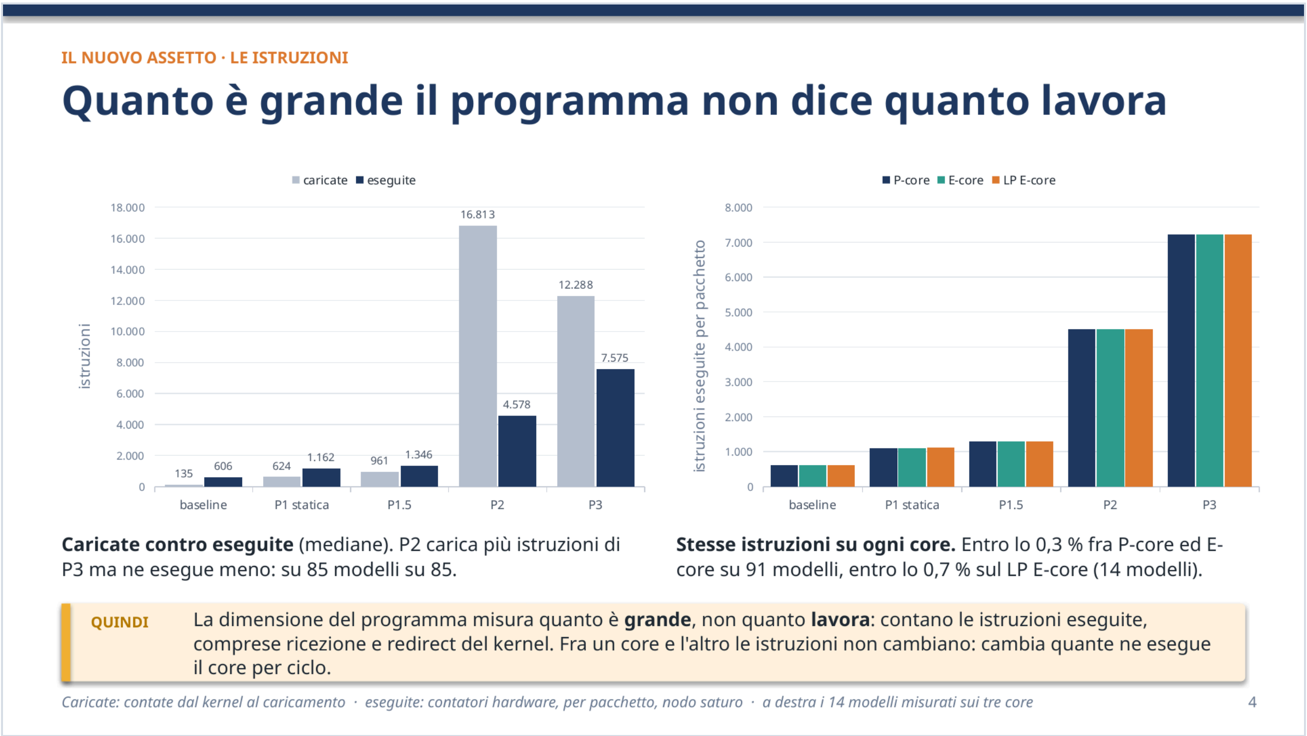
Le schede a destra danno l'intervallo dal modello più leggero al più pesante: il tempo per pacchetto e la capacità.

Esempio di calcolo. La mediana di P2 regge 3,4 milioni di pacchetti al secondo: spende 1 / 3 400 000 s ≈ 291 ns per pacchetto. La baseline ne spende 67. La differenza, ~225 ns, è il tempo che P2 dedica alla rete neurale.

La rete neurale costa 19–155 ns con i pesi scritti nel programma (P1; mediana 52), 129–446 ns con la forma letta da tabella (P2; mediana 225), 240–852 ns con anche la profondità (P3; mediana 420). Un core regge da 11,7 a 1,1 milioni di pacchetti al secondo, secondo modello e pipeline. L'ordine P1 ≤ P2 ≤ P3 regge su 85 modelli su 85.

SLIDE 4

Le istruzioni: caricate contro eseguite



IL NUOVO ASSETTO · LE ISTRUZIONI

Quanto è grande il programma non dice quanto lavora

Caricate contro eseguite (mediane). P2 carica più istruzioni di P3 ma ne esegue meno: su 85 modelli su 85.

Stesse istruzioni su ogni core. Entro lo 0,3 % fra P-core ed E-core su 91 modelli, entro lo 0,7 % sul LP E-core (14 modelli).

QUINDI La dimensione del programma misura quanto è **grande**, non quanto **lavora**: contano le istruzioni eseguite, comprese ricezione e redirect del kernel. Fra un core e l'altro le istruzioni non cambiano: cambia quante ne esegue il core per ciclo.

Caricate: contate dal kernel al caricamento · eseguite: contatori hardware, per pacchetto, nodo saturo · a destra i 14 modelli misurati sui tre core

Grafico a sinistra. Due modi di contare le istruzioni, in mediana sulla popolazione:

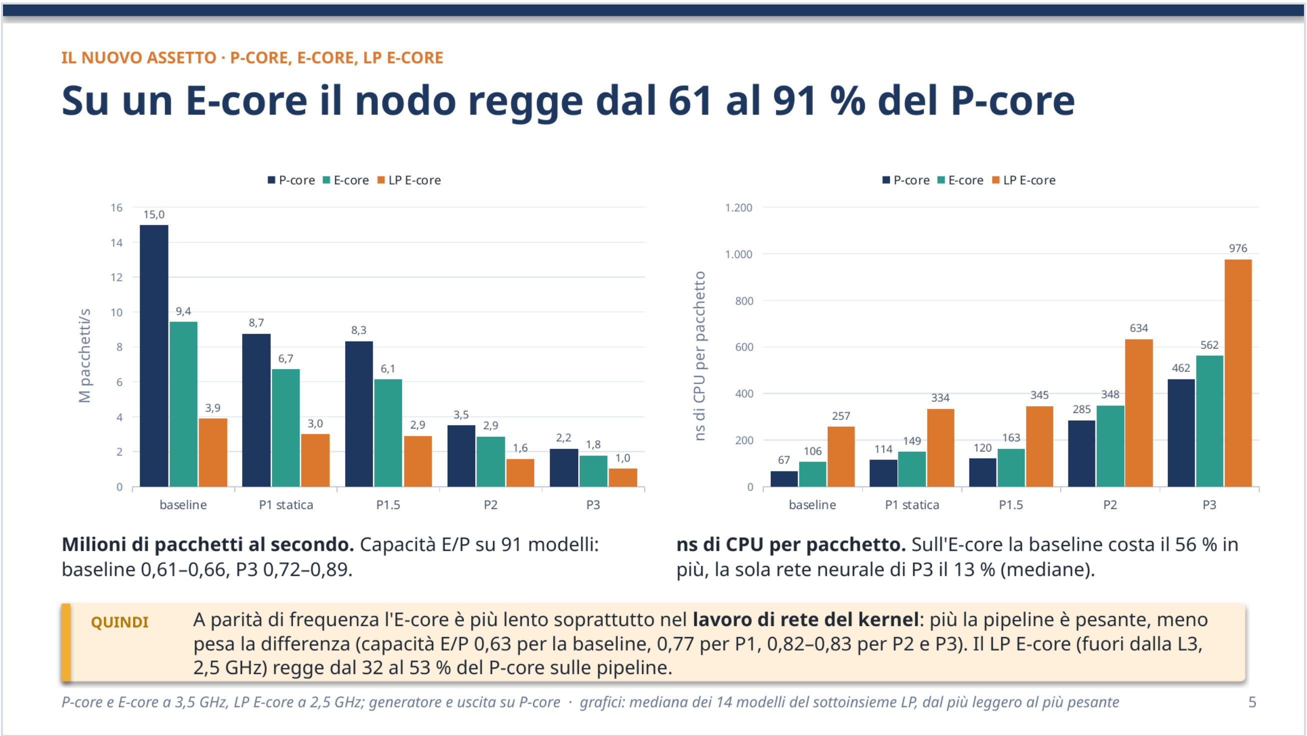
- **caricate** (grigio): quante istruzioni ha il programma, contate dal kernel quando lo carica. È la dimensione del programma;
- **eseguite** (blu): quante istruzioni il core del nodo esegue davvero per ogni pacchetto, contate dal processore stesso con i contatori hardware durante la misura sotto traffico. Comprendono anche il lavoro del kernel attorno al programma (ricevere, inoltrare).

Le due misure dicono cose diverse. P2 è il programma più grande (14 604–18 937 istruzioni, mediana 16 813) ma ne esegue meno di P3 (mediana 4 578 contro 7 575): srotola cicli fino ai valori massimi, ma molti rami non vengono percorsi. P3 è più piccolo (12 288, uguale per ogni modello) ma ne esegue di più: rientra in sé stesso una volta per strato. Succede su **85 modelli su 85**. **La dimensione non predice il lavoro** fra pipeline diverse; dentro una pipeline ordina bene i modelli per P1 (correlazione di rango 0,98), meno per P2 (0,77).

Grafico a destra. Le istruzioni eseguite per pacchetto sul P-core, sull'E-core e sul LP E-core, sui 14 modelli misurati su tutti e tre: le barre coincidono. Sui 91 modelli P-core ed E-core differiscono al più dello 0,3 %, il LP E-core dello 0,7 % sulle pipeline. Il codice è lo stesso; è la base per la slide 6: se i core vanno a velocità diverse, non è perché eseguono istruzioni diverse.

SLIDE 5

Il nuovo assetto: P-core, E-core, LP E-core



IL NUOVO ASSETTO · P-CORE, E-CORE, LP E-CORE

Su un E-core il nodo regge dal 61 al 91 % del P-core

Milioni di pacchetti al secondo. Capacità E/P su 91 modelli: baseline 0,61–0,66, P3 0,72–0,89.

ns di CPU per pacchetto. Sull'E-core la baseline costa il 56 % in più, la sola rete neurale di P3 il 13 % (mediane).

QUINDI A parità di frequenza l'E-core è più lento soprattutto nel **lavoro di rete del kernel**: più la pipeline è pesante, meno pesa la differenza (capacità E/P 0,63 per la baseline, 0,77 per P1, 0,82–0,83 per P2 e P3). Il LP E-core (fuori dalla L3, 2,5 GHz) regge dal 32 al 53 % del P-core sulle pipeline.

P-core e E-core a 3,5 GHz, LP E-core a 2,5 GHz; generatore e uscita su P-core · grafici: mediana dei 14 modelli del sottoinsieme LP, dal più leggero al più pesante

Il processore del portatile ha tre tipi di core: sei **P-core** veloci, otto **E-core** efficienti (in due moduli da quattro che dividono cache e frequenza) e due **LP E-core** a basso consumo, su un'altra parte del chip. La domanda: che cosa succede se il nodo gira su un core lento?

Il disegno della misura: cambia solo il core del nodo. Il generatore e il nodo successivo restano su P-core, così il generatore satura sempre il nodo. L'E-core gira alla stessa frequenza del P-core, 3,5 GHz, quindi la differenza dipende solo da come il core esegue il codice. Il LP E-core gira al suo massimo, 2,5 GHz. Gli altri tre core del modulo dell'E-core restano a riposo.

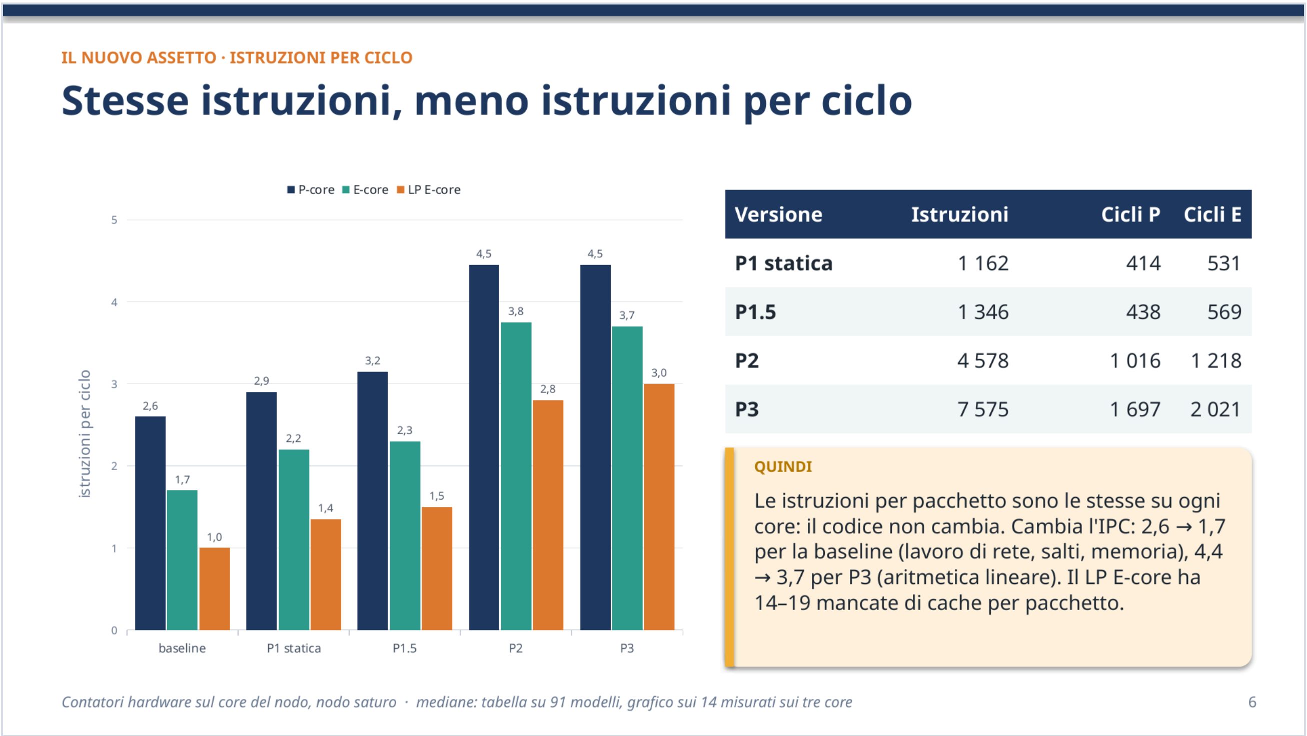
A sinistra i milioni di pacchetti al secondo, **a destra** i nanosecondi per pacchetto: mediane dei 14 modelli misurati su tutti e tre i core, dal più leggero al più pesante.

Sui 91 modelli l'E-core regge, rispetto al P-core, 0,63 [0,61–0,66] per la baseline, 0,77 [0,70–0,91] per P1, 0,82 [0,77–0,87] per P2 e 0,83 [0,72–0,89] per P3. Il LP E-core regge dal 32 al 53 % del P-core sulle pipeline.

Esempio. P3, mediana dei 14 modelli: 2,2 milioni al secondo sul P-core, 1,8 sull'E-core, 1,0 sul LP E-core. In tempo: 462, 562 e 976 ns per pacchetto.

SLIDE 6

Stesse istruzioni, meno istruzioni per ciclo



IL NUOVO ASSETTO · ISTRUZIONI PER CICLO

Stesse istruzioni, meno istruzioni per ciclo

Versione	Istruzioni	Cicli P	Cicli E
P1 statica	1 162	414	531
P1.5	1 346	438	569
P2	4 578	1 016	1 218
P3	7 575	1 697	2 021

QUINDI

Le istruzioni per pacchetto sono le stesse su ogni core: il codice non cambia. Cambia l'IPC: 2,6 → 1,7 per la baseline (lavoro di rete, salti, memoria), 4,4 → 3,7 per P3 (aritmetica lineare). Il LP E-core ha 14–19 mancate di cache per pacchetto.

Contatori hardware sul core del nodo, nodo saturo · mediane: tabella su 91 modelli, grafico sui 14 misurati sui tre core

Se le istruzioni per pacchetto sono le stesse (slide 4), la differenza fra i core sta nei **cicli**: quanti battiti d'orologio servono per eseguirle. Il rapporto fra istruzioni e cicli è l'**IPC**, istruzioni per ciclo.

Il grafico mostra l'IPC sui tre core (mediane dei 14 modelli misurati sui tre); la tabella istruzioni e cicli per pacchetto su P-core ed E-core (mediane dei 91).

> **Esempio.** La mediana di P3 esegue 7 575 istruzioni per pacchetto. Il P-core ci mette 1 697 cicli (IPC 4,4), l'E-core 2 021 (IPC 3,7). A 3,5 GHz sono circa 485 e 577 ns: lo stesso ordine dei tempi di CPU misurati, quindi i due metodi di misura concordano.

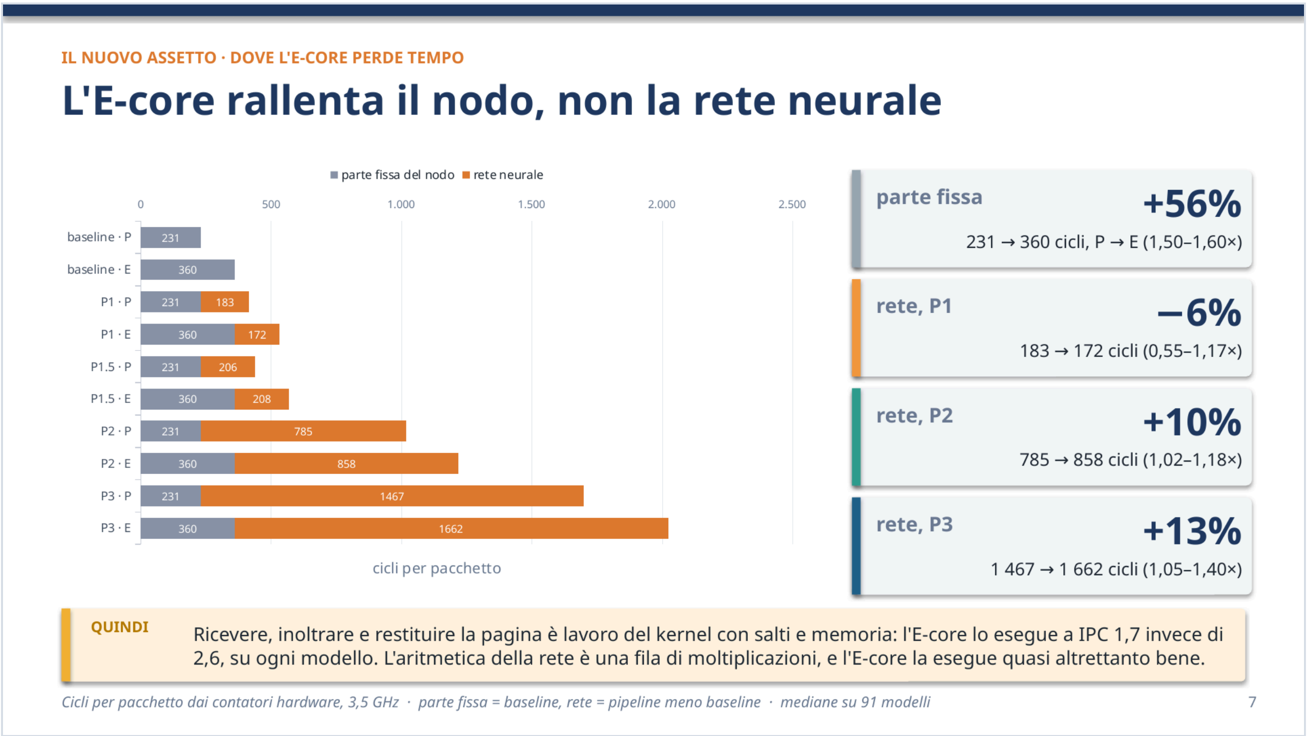
Due cose si leggono nel grafico:

- l'IPC cresce con la pipeline: la baseline (lavoro del kernel, salti, accessi alla memoria) sta a 2,6 sul P-core, P2 e P3 (aritmetica in fila, senza bivi) a 4,4–4,5;
- il distacco fra i core è più grande dove l'IPC è basso: 2,6 → 1,7 per la baseline, 4,4 → 3,7 per P3. Modello per modello, l'IPC dell'E-core sta a 0,65 [0,63–0,68] di quello del P-core per la baseline e a 0,84 [0,70–0,89] per P3.

Il LP E-core ha l'IPC più basso di tutti (1,0–3,0) perché, oltre al core più semplice, aspetta la memoria (slide 8).

SLIDE 7

Dove l'E-core perde tempo



IL NUOVO ASSETTO · DOVE L'E-CORE PERDE TEMPO

L'E-core rallenta il nodo, non la rete neurale

parte fissa **+56%**
231 → 360 cicli, P → E (1,50–1,60×)

rete, P1 **−6%**
183 → 172 cicli (0,55–1,17×)

rete, P2 **+10%**
785 → 858 cicli (1,02–1,18×)

rete, P3 **+13%**
1 467 → 1 662 cicli (1,05–1,40×)

QUINDI Ricevere, inoltrare e restituire la pagina è lavoro del kernel con salti e memoria: l'E-core lo esegue a IPC 1,7 invece di 2,6, su ogni modello. L'aritmetica della rete è una fila di moltiplicazioni, e l'E-core la esegue quasi altrettanto bene.

Cicli per pacchetto dai contatori hardware, 3,5 GHz · parte fissa = baseline, rete = pipeline meno baseline · mediane su 91 modelli 7

Le barre dividono i cicli per pacchetto in due parti, sul P-core (P) e sull'E-core (E), in mediana sui 91 modelli:

- **parte fissa del nodo** (grigio): i cicli della baseline, cioè ricevere, leggere l'intestazione, inoltrare, restituire la memoria del pacchetto;
- **rete neurale** (arancione): i cicli che la pipeline aggiunge sopra la baseline.

Il risultato è netto. La parte fissa passa da 231 a 360 cicli: +56 %, fra il 50 e il 61 % su ogni modello. La rete neurale cambia poco: modello per modello, i suoi cicli sull'E-core stanno a 0,94 [0,55–1,17] volte quelli del P-core per P1, 1,10 [1,02–1,18] per P2, 1,13 [1,05–1,40] per P3.

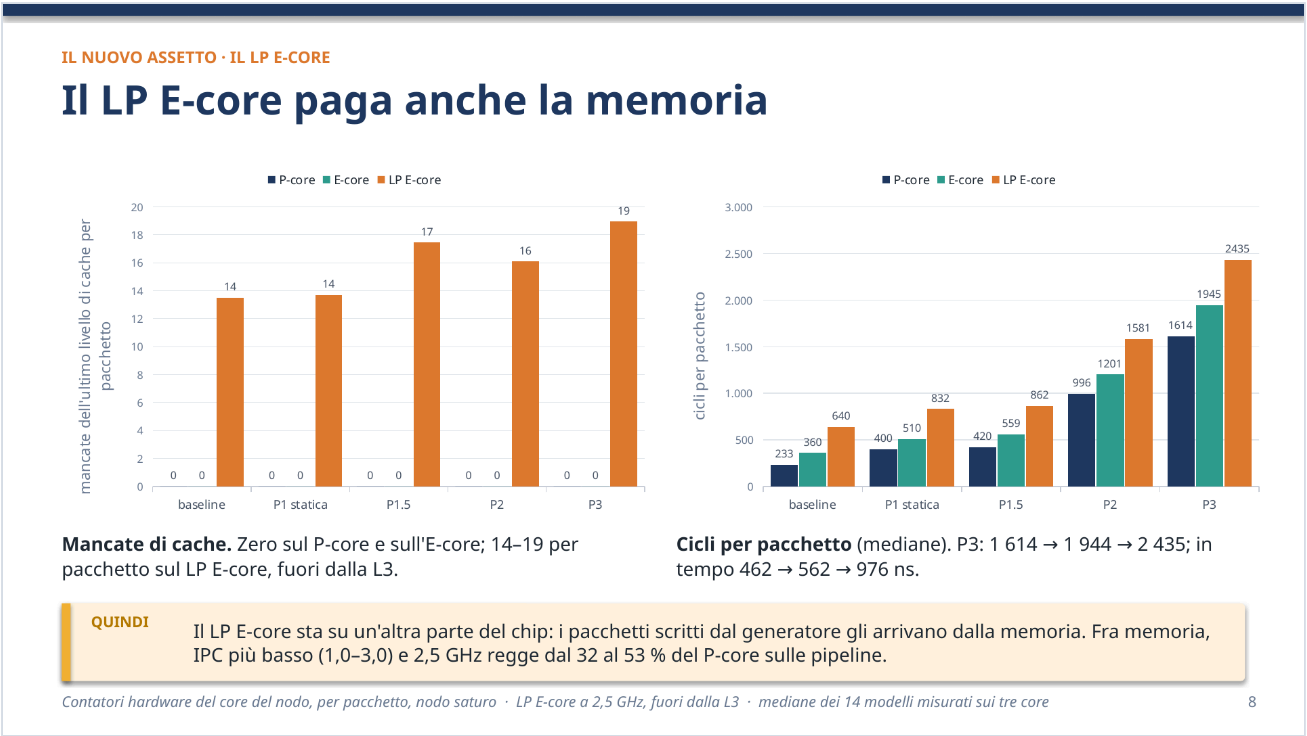
Perché. La parte fissa è codice del kernel con molti salti e accessi alla memoria, dove un core più semplice si ferma spesso ad aspettare. La rete neurale è una fila di moltiplicazioni e somme indipendenti, che anche l'E-core esegue molte per volta.

È questa la ragione per cui il rapporto di capacità della slide 5 sale con il peso della pipeline: più una pipeline passa tempo nella rete neurale, meno conta che il core sia un E-core. Le misure sono in cicli, quindi non dipendono dalla frequenza.

P1 sotto l'1. Per P1 la rete costa poche decine di cicli sopra una baseline di 231–360: la differenza fra due numeri grandi è rumorosa, e il rapporto E/P va da 0,55 a 1,17. Vale come «circa uguale», non come «l'E-core è più veloce».

SLIDE 8

Il LP E-core paga anche la memoria



IL NUOVO ASSETTO · IL LP E-CORE

Il LP E-core paga anche la memoria

Mancate di cache. Zero sul P-core e sull'E-core; 14–19 per pacchetto sul LP E-core, fuori dalla L3.

Cicli per pacchetto (mediane). P3: 1 614 → 1 944 → 2 435; in tempo 462 → 562 → 976 ns.

QUINDI Il LP E-core sta su un'altra parte del chip: i pacchetti scritti dal generatore gli arrivano dalla memoria. Fra memoria, IPC più basso (1,0–3,0) e 2,5 GHz regge dal 32 al 53 % del P-core sulle pipeline.

Contatori hardware del core del nodo, per pacchetto, nodo saturo · LP E-core a 2,5 GHz, fuori dalla L3 · mediane dei 14 modelli misurati sui tre core

A sinistra le mancate dell'ultimo livello di cache per pacchetto: quante volte il core cerca un dato nella cache più grande e non lo trova, e deve andare in memoria. Sul P-core e sull'E-core sono zero: i pacchetti che il generatore scrive restano nella cache L3, condivisa da P-core ed E-core. Il LP E-core sta fuori dalla L3: ne ha 14–19 per pacchetto (mediane dei 14 modelli), e ogni volta aspetta la memoria.

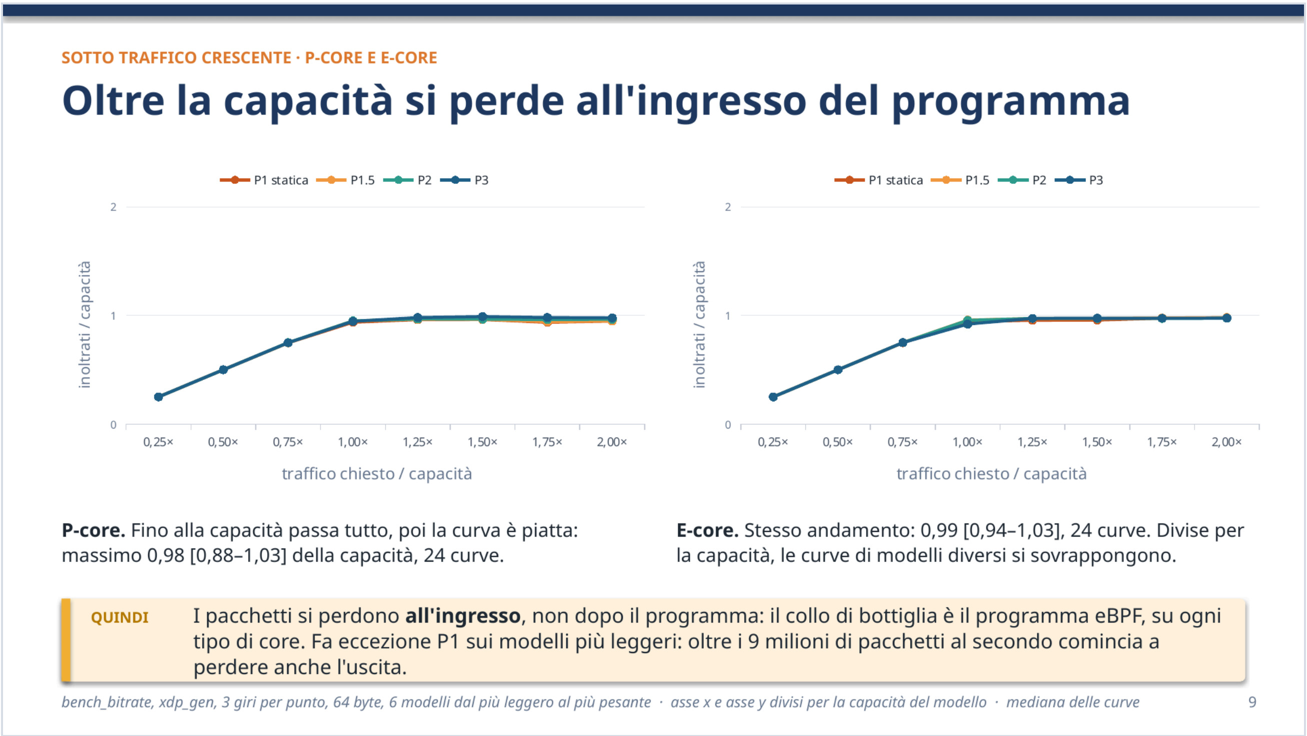
A destra i cicli per pacchetto sui tre core. Per P3: 1 614 sul P-core, 1 945 sull'E-core, 2 435 sul LP E-core. In tempo pesa anche la frequenza più bassa del LP E-core (2,5 GHz): 462, 562 e 976 ns.

Esempio. La baseline sul LP E-core spende 640 cicli per pacchetto contro 233 sul P-core: quasi tre volte tanto, per lo stesso codice. La differenza è quasi tutta attesa della memoria.

Il LP E-core regge dal 32 al 53 % del P-core sulle pipeline: dal 35 % per P1 al 48 % per P3, in mediana. È il caso estremo: un core pensato per tenere acceso il sistema consumando poco, non per elaborare pacchetti.

SLIDE 9

Oltre la capacità si perde all'ingresso del programma



SOTTO TRAFFICO CRESCENTE · P-CORE E E-CORE

Oltre la capacità si perde all'ingresso del programma

P-core. Fino alla capacità passa tutto, poi la curva è piatta: massimo 0,98 [0,88–1,03] della capacità, 24 curve.

E-core. Stesso andamento: 0,99 [0,94–1,03], 24 curve. Divise per la capacità, le curve di modelli diversi si sovrappongono.

QUINDI I pacchetti si perdono **all'ingresso**, non dopo il programma: il collo di bottiglia è il programma eBPF, su ogni tipo di core. Fa eccezione P1 sui modelli più leggeri: oltre i 9 milioni di pacchetti al secondo comincia a perdere anche l'uscita.

bench_bitrate, xdp_gen, 3 giri per punto, 64 byte, 6 modelli dal più leggero al più pesante · asse x e asse y divisi per la capacità del modello · mediana delle curve

Qui il traffico **cresce** gradualmente, da 0,5 a 12 milioni di pacchetti al secondo, e si contano i pacchetti inoltrati. A sinistra il nodo su un P-core, a destra su un E-core, sei modelli dal più leggero al più pesante.

Modelli diversi hanno capacità diverse. Per metterli sullo stesso grafico, ogni curva è divisa per la capacità del suo modello: sull'asse x il traffico chiesto diviso la capacità, sull'asse y gli inoltrati divisi la capacità. Le linee sono la mediana delle curve. Così 1 vuol dire «la capacità di quel modello», qualunque sia.

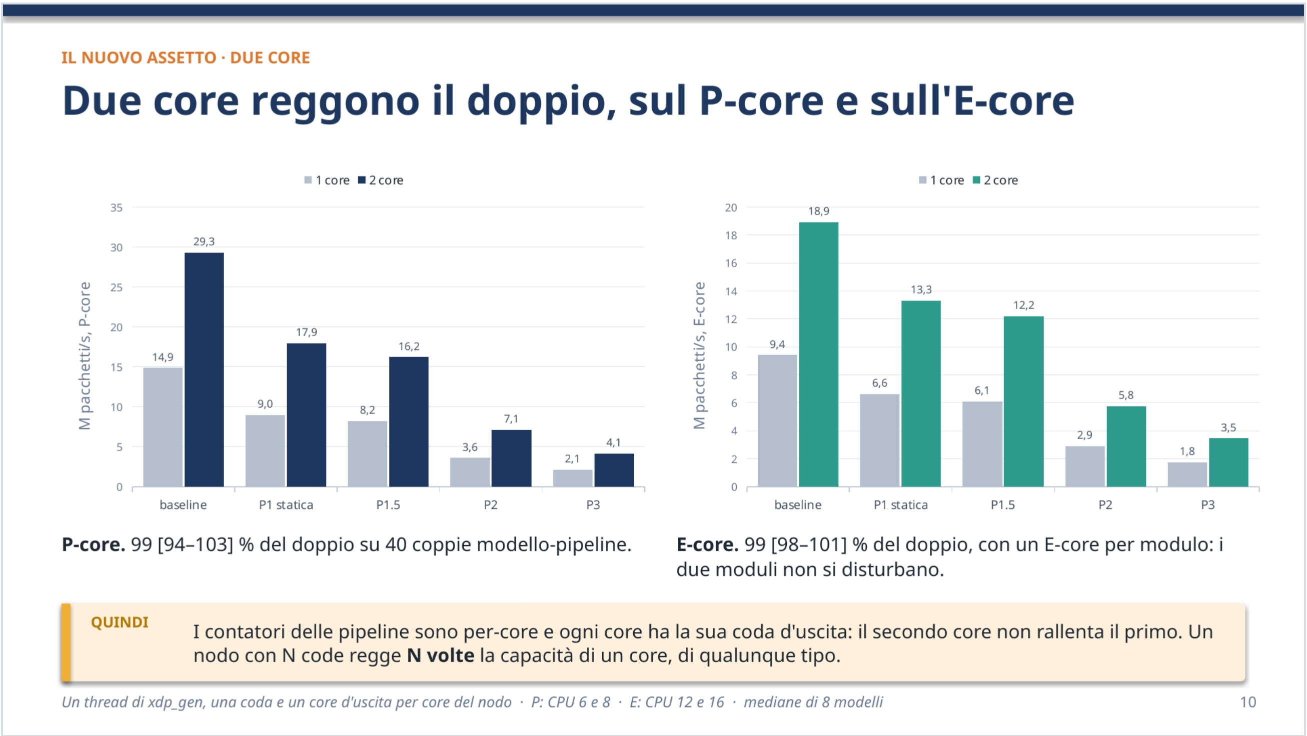
Ogni curva sale lungo la diagonale (tutto ciò che arriva viene inoltrato) finché il traffico resta sotto la capacità, poi diventa piatta: il nodo non può inoltrare di più, e il resto viene respinto dalla coda d'ingresso piena. Il massimo inoltrato sta a 0,98 [0,88–1,03] della capacità misurata a massima spinta sul P-core e a 0,99 [0,94–1,03] sull'E-core, su 24 curve per core.

Dove si perde. Il banco conta i pacchetti in tre punti: inviati, ricevuti dal programma, inoltrati. Della perdita, la parte persa **prima** del programma, alla coda, è 1,000 in mediana: il programma rilancia tutto ciò che riceve. Il collo di bottiglia è il programma eBPF, non la trasmissione, su ogni tipo di core.

L'eccezione. P1 sui modelli più leggeri regge oltre 9 milioni di pacchetti al secondo. Lì comincia a perdere anche l'uscita: la quota persa alla coda scende fino a 0,69, e il massimo inoltrato a 0,88 della capacità. È il limite del cavo virtuale e del core d'uscita, non della rete neurale.

SLIDE 10

Due core reggono il doppio



IL NUOVO ASSETTO · DUE CORE

Due core reggono il doppio, sul P-core e sull'E-core

P-core. 99 [94–103] % del doppio su 40 coppie modello-pipeline.

E-core. 99 [98–101] % del doppio, con un E-core per modulo: i due moduli non si disturbano.

QUINDI I contatori delle pipeline sono per-core e ogni core ha la sua coda d'uscita: il secondo core non rallenta il primo. Un nodo con N code regge **N volte** la capacità di un core, di qualunque tipo.

Un thread di xdp_gen, una coda e un core d'uscita per core del nodo · P: CPU 6 e 8 · E: CPU 12 e 16 · mediane di 8 modelli

Il nodo riceve **due core**, ciascuno con la sua coda d'ingresso alimentata da un thread del generatore, la sua coda d'uscita e il suo core d'uscita. A sinistra due P-core, a destra due E-core (uno per modulo). Le barre chiare sono un core, quelle scure due: mediane degli 8 modelli del sottoinsieme, dal più leggero al più pesante.

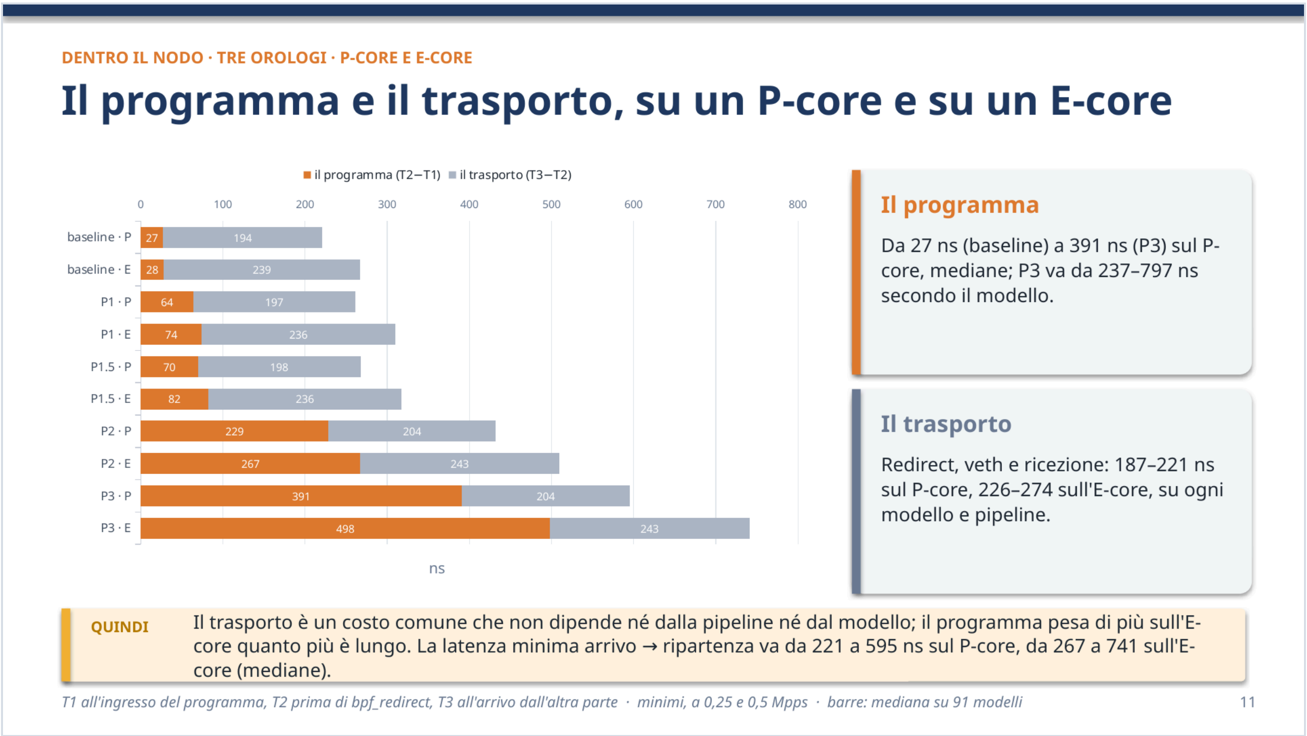
Come si legge. P2 con un P-core regge 3,6 milioni al secondo in mediana, con due 7,1: il doppio. Sull'E-core 2,9 e 5,8.

Due core reggono il 99 [94–103] % del doppio di un core sul P-core e il 99 [98–101] % sull'E-core, su 40 coppie modello-pipeline per core. Il secondo core non rallenta il primo perché non condividono niente di scritto a ogni pacchetto: i contatori delle pipeline sono per-core, e ogni core ha la sua coda d'uscita.

Conseguenza pratica: una scheda di rete vera distribuisce i pacchetti su più code e più core (RSS). Un nodo con N code regge circa N volte la capacità misurata su un core, con qualunque modello.

SLIDE 11

Tre orologi: il programma e il trasporto



DENTRO IL NODO · TRE OROLOGI · P-CORE E E-CORE

Il programma e il trasporto, su un P-core e su un E-core

Il programma

Da 27 ns (baseline) a 391 ns (P3) sul P-core, mediane; P3 va da 237–797 ns secondo il modello.

Il trasporto

Redirect, veth e ricezione: 187–221 ns sul P-core, 226–274 sull'E-core, su ogni modello e pipeline.

QUINDI Il trasporto è un costo comune che non dipende né dalla pipeline né dal modello; il programma pesa di più sull'E-core quanto più è lungo. La latenza minima arrivo → ripartenza va da 221 a 595 ns sul P-core, da 267 a 741 sull'E-core (mediane).

T1 all'ingresso del programma, T2 prima di bpf_redirect, T3 all'arrivo dall'altra parte · minimi, a 0,25 e 0,5 Mpps · barre: mediana su 91 modelli

Con una versione del programma che legge l'orologio in tre punti si separa il tempo del programma da quello del trasporto:

- **T1** all'ingresso del programma, **T2** subito prima di inoltrare: T2−T1 è il programma (arancione);
- **T3** all'arrivo al nodo successivo: T3−T2 è il trasporto, cioè redirect, cavo virtuale e ricezione (grigio).

Le barre mostrano i minimi di ogni pezzo, in mediana sui 91 modelli, per ogni pipeline sul P-core (P) e sull'E-core (E).

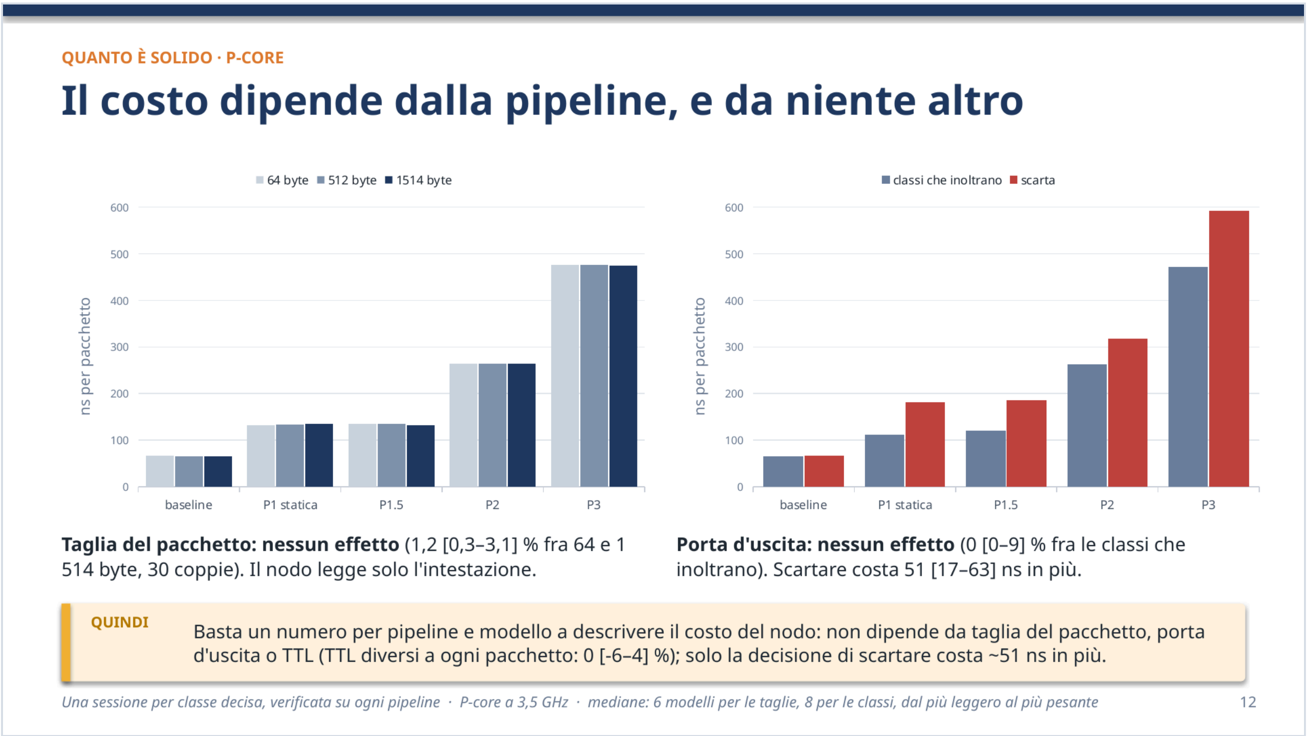
Il trasporto è un costo comune: 187–221 ns sul P-core e 226–274 sull'E-core su ogni modello e pipeline; fra la pipeline più cara e la meno cara dello stesso modello differisce del 4 [1–17] %. Il programma va da 27 ns (baseline) a 391 ns (P3) sul P-core in mediana, e per P3 da 237 a 797 ns secondo il modello; sull'E-core costa di più, tanto di più quanto più è lungo (P3: 498 ns in mediana).

Esempio. La mediana di P2 sul P-core: 229 ns di programma più 204 di trasporto. La somma, ~430 ns, è la latenza minima dall'arrivo del pacchetto alla sua ripartenza.

Il minimo, non la media. Qui l'uscita gira sullo stesso core del nodo, quindi la media di T3−T2 contiene anche l'attesa in coda e cresce col traffico (da ~550 ns a 0,25 milioni di pacchetti al secondo a ~1 200 a 0,5). Il minimo è il trasporto senza attesa. Questa versione legge l'orologio due volte in più per pacchetto: serve a capire dove va il tempo, non a misurare quanti pacchetti regge il nodo (quello lo fanno le slide 3 e 5).

SLIDE 12

Il costo dipende dalla pipeline, e da niente altro



QUANTO È SOLIDO · P-CORE

Il costo dipende dalla pipeline, e da niente altro

Taglia del pacchetto: nessun effetto (1,2 [0,3–3,1] % fra 64 e 1 514 byte, 30 coppie). Il nodo legge solo l'intestazione.

Porta d'uscita: nessun effetto (0 [0–9] % fra le classi che inoltrano). Scartare costa 51 [17–63] ns in più.

QUINDI Basta un numero per pipeline e modello a descrivere il costo del nodo: non dipende da taglia del pacchetto, porta d'uscita o TTL (TTL diversi a ogni pacchetto: 0 [-6–4] %); solo la decisione di scartare costa ~51 ns in più.

Una sessione per classe decisa, verificata su ogni pipeline · P-core a 3,5 GHz · mediane: 6 modelli per le taglie, 8 per le classi, dal più leggero al più pesante 12

Tre controlli sul P-core, ciascuno su un sottoinsieme dal modello più leggero al più pesante:

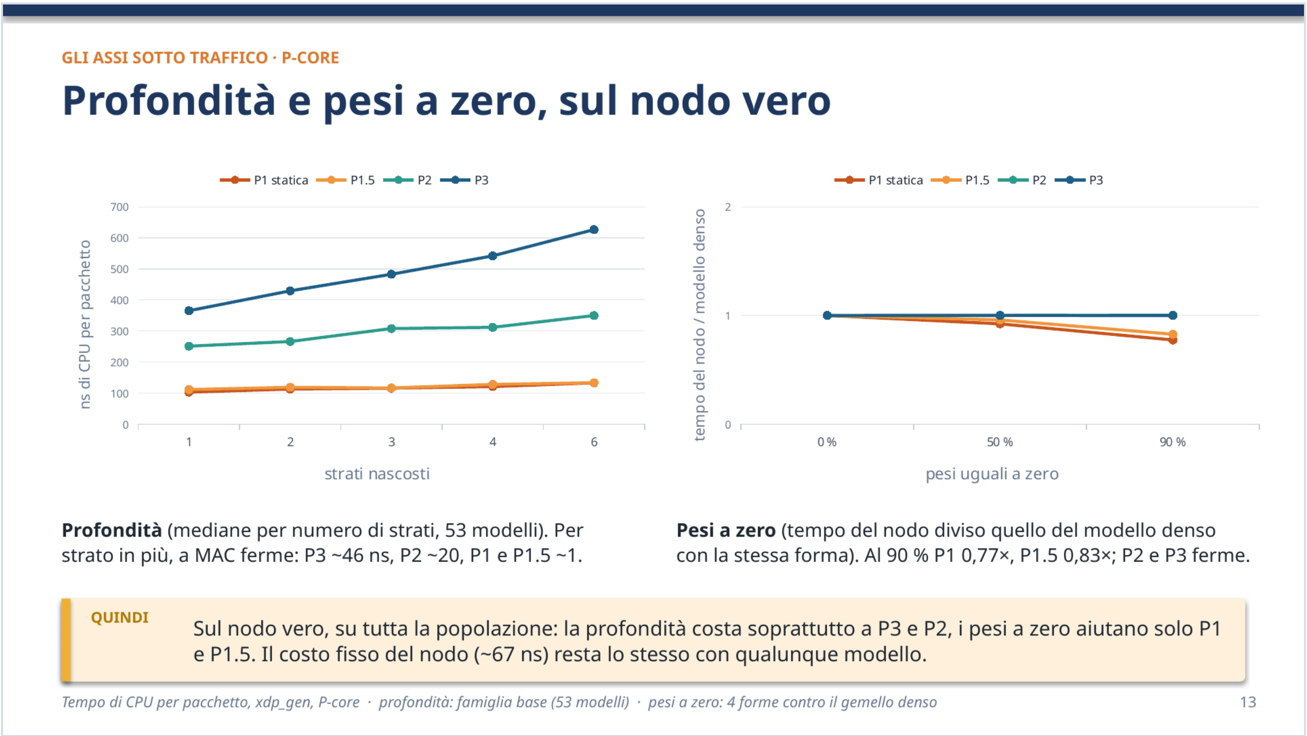
- **a sinistra** la taglia del pacchetto (6 modelli): a 64, 512 e 1 514 byte ogni pipeline costa uguale, con una differenza di 1,2 [0,3–3,1] % fra la taglia più cara e la meno cara. Il nodo legge solo l'intestazione, come farebbe su una scheda di rete vera;
- **a destra** la classe decisa (8 modelli): stato dei collegamenti e TTL sono scelti apposta perché la rete decida ciascuna delle classi che il modello sa raggiungere. Le classi che inoltrano (su porte diverse) costano uguale, 0 [0–9] % fra la più cara e la meno cara; la classe che **scarta** (rosso) costa 51 [17–63] ns in più;
- nel riquadro, i **TTL diversi a ogni pacchetto** (12 modelli): il modello decide classi diverse pacchetto per pacchetto, e il tempo del nodo cambia di 0 [−6–4] % rispetto al pacchetto fisso.

Perché scartare costa di più. Quando il pacchetto viene inoltrato, la sua memoria si libera sul core d'uscita; quando viene scartato, si libera sul core del nodo, che quindi lavora di più. È un effetto del banco, non della rete neurale.

Basta quindi un numero per pipeline e modello a descrivere il costo del nodo: non dipende dalla taglia del pacchetto né dalla porta d'uscita. Il traffico in bit al secondo che il nodo regge cresce con la taglia del pacchetto.

SLIDE 13

Profondità e pesi a zero, sul nodo vero



GLI ASSI SOTTO TRAFFICO · P-CORE

Profondità e pesi a zero, sul nodo vero

Profondità (mediane per numero di strati, 53 modelli). Per strato in più, a MAC ferme: P3 ~46 ns, P2 ~20, P1 e P1.5 ~1.

Pesi a zero (tempo del nodo diviso quello del modello denso con la stessa forma). Al 90 % P1 0,77×, P1.5 0,83×; P2 e P3 ferme.

QUINDI Sul nodo vero, su tutta la popolazione: la profondità costa soprattutto a P3 e P2, i pesi a zero aiutano solo P1 e P1.5. Il costo fisso del nodo (~67 ns) resta lo stesso con qualunque modello.

Tempo di CPU per pacchetto, xdp_gen, P-core · profondità: famiglia base (53 modelli) · pesi a zero: 4 forme contro il gemello denso

Qui il tempo è quello del **nodo intero** (rete neurale più costo fisso), sul P-core.

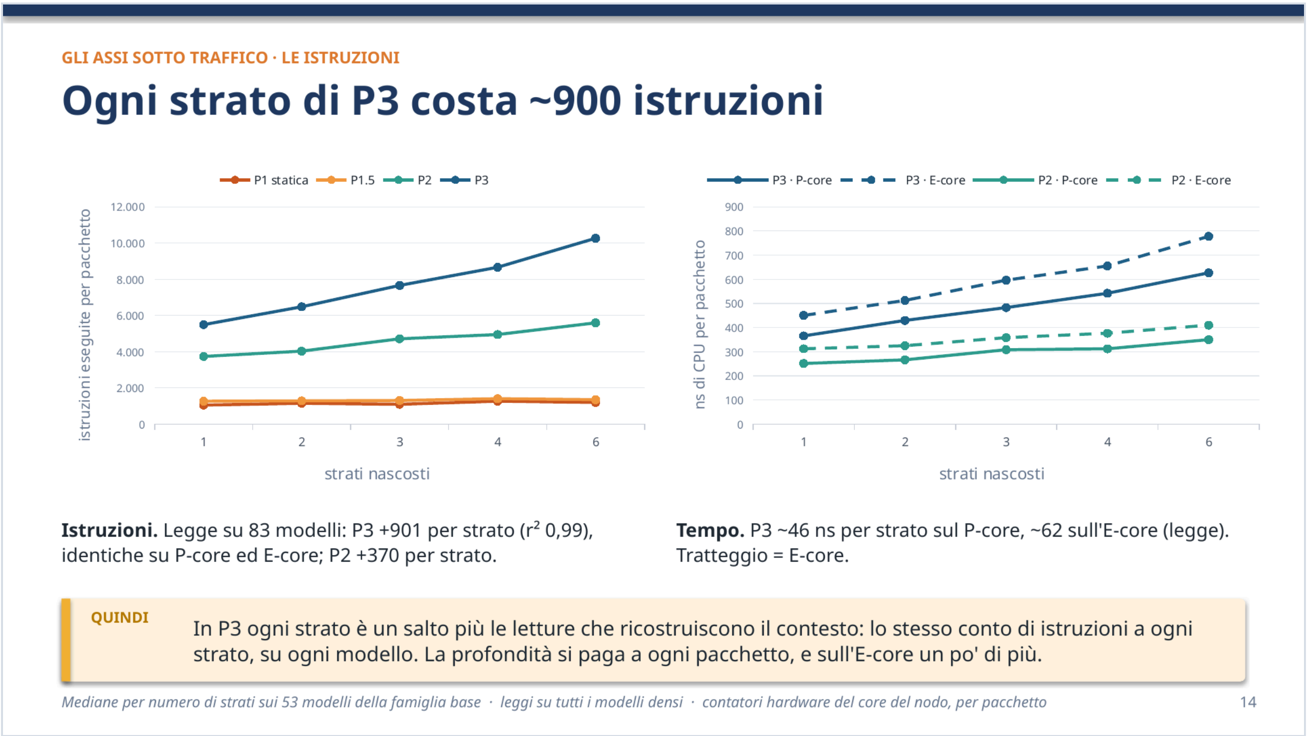
A sinistra la profondità: la mediana del tempo per pacchetto per numero di strati nascosti, sui 53 modelli della famiglia base (che hanno 1, 2, 3, 4 o 6 strati). Ogni punto mescola composizioni e larghezze diverse; la legge separa l'effetto dello strato: a moltiplicazioni ferme, uno strato in più costa circa 46 ns a P3, 20 a P2, 1 a P1 e P1.5.

A destra i pesi a zero: per le quattro forme con il 50 e il 90 % dei pesi a zero, il tempo del nodo diviso quello del modello denso con la stessa forma e composizione. P1 e P1.5 accelerano (al 90 %: 0,77× e 0,83×), perché il compilatore vede lo zero e cancella la moltiplicazione; P2 e P3 non cambiano, perché per loro uno zero letto da una tabella è un numero come un altro.

Le curve sono quelle della rete neurale (slide 19–23) più il costo fisso del nodo, ~67 ns con qualunque modello.

SLIDE 14

Ogni strato di P3 costa ~900 istruzioni



GLI ASSI SOTTO TRAFFICO · LE ISTRUZIONI

Ogni strato di P3 costa ~900 istruzioni

Istruzioni. Legge su 83 modelli: P3 +901 per strato (r^2 0,99), identiche su P-core ed E-core; P2 +370 per strato.

Tempo. P3 ~46 ns per strato sul P-core, ~62 sull'E-core (legge). Tratteggio = E-core.

QUINDI In P3 ogni strato è un salto più le letture che ricostruiscono il contesto: lo stesso conto di istruzioni a ogni strato, su ogni modello. La profondità si paga a ogni pacchetto, e sull'E-core un po' di più.

Mediane per numero di strati sui 53 modelli della famiglia base · leggi su tutti i modelli densi · contatori hardware del core del nodo, per pacchetto

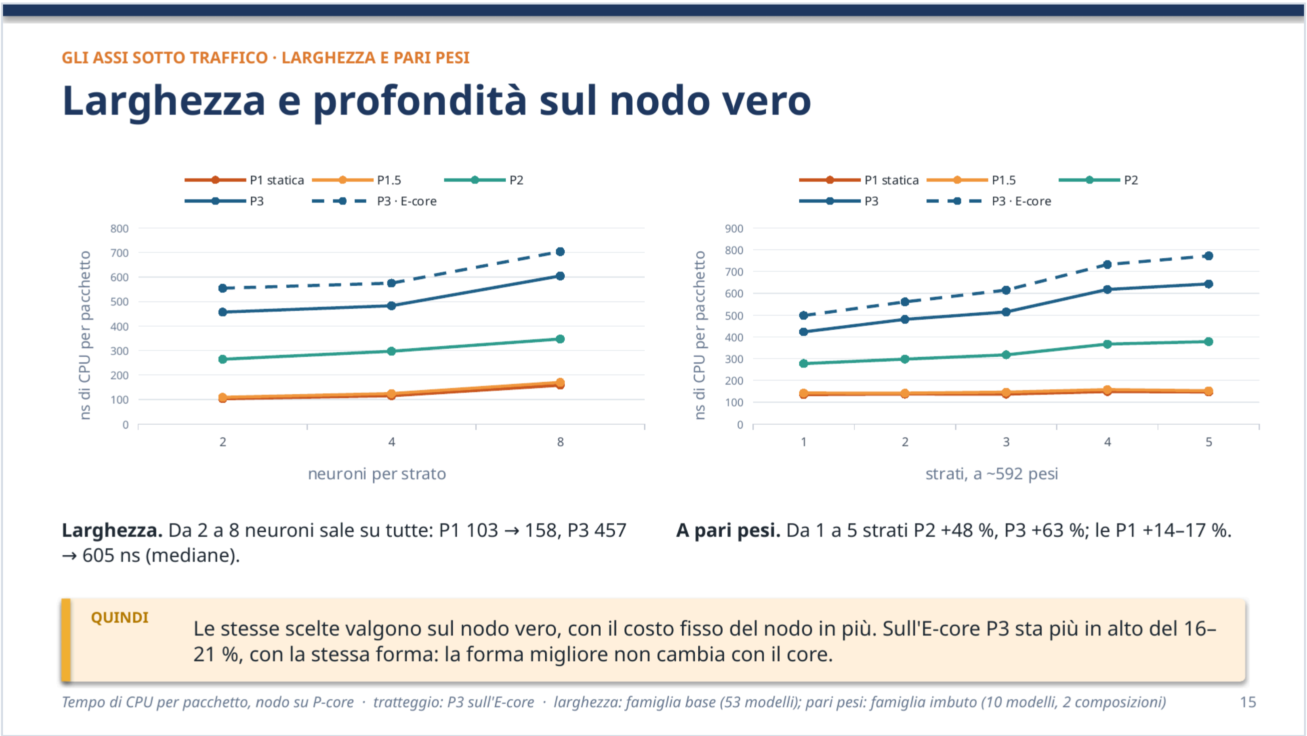
A sinistra le istruzioni eseguite per pacchetto, dai contatori hardware, in mediana per numero di strati (famiglia base). La legge stimata su tutti gli 83 modelli densi dà a P3 **+901 istruzioni per ogni strato** (r^2 0,99), identiche su P-core ed E-core; a P2 +370; P1 e P1.5 non pagano lo strato (le loro istruzioni seguono le moltiplicazioni, non gli strati).

Che cosa sono le ~900 istruzioni. P3 esegue uno strato per volta: alla fine di ogni strato salta di nuovo al programma generico dello strato, che deve rileggere da tabelle dove si trova, che forma ha lo strato e dove sono i suoi pesi. Il conto è lo stesso su ogni modello, ed è per questo che il costo per strato è una costante della legge.

A destra il tempo per pacchetto di P3 e P2, sul P-core (linea continua) e sull'E-core (tratteggio), in mediana per numero di strati. Per la legge P3 paga circa 46 ns per strato sul P-core e 62 sull'E-core: stesse istruzioni, eseguite a un IPC più basso.

SLIDE 15

Larghezza e profondità sul nodo vero



GLI ASSI SOTTO TRAFFICO · LARGHEZZA E PARI PESI

Larghezza e profondità sul nodo vero

Larghezza. Da 2 a 8 neuroni sale su tutte: P1 103 → 158, P3 457 → 605 ns (mediane).

A pari pesi. Da 1 a 5 strati P2 +48 %, P3 +63 %; le P1 +14–17 %.

QUINDI Le stesse scelte valgono sul nodo vero, con il costo fisso del nodo in più. Sull'E-core P3 sta più in alto del 16–21 %, con la stessa forma: la forma migliore non cambia con il core.

Tempo di CPU per pacchetto, nodo su P-core · tratteggio: P3 sull'E-core · larghezza: famiglia base (53 modelli); pari pesi: famiglia imbuto (10 modelli, 2 composizioni)

Altre due analisi sotto traffico vero, sul P-core, nel tempo del nodo intero; il tratteggio è P3 sull'E-core.

A sinistra la larghezza: mediana per numero di neuroni per strato (2, 4, 8) sulla famiglia base. Sale su tutte le pipeline: P1 103 → 158 ns, P3 457 → 605. Un neurone in più è lavoro vero a ogni pacchetto.

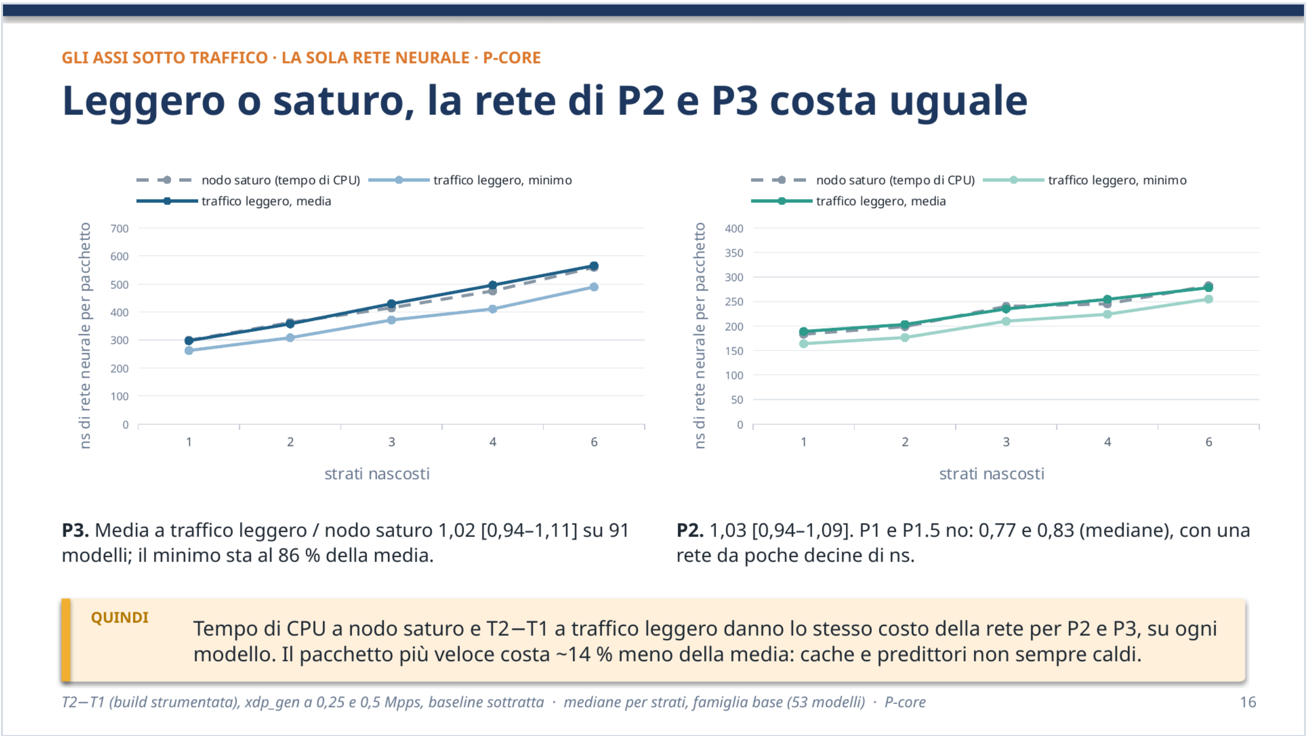
A destra a pari pesi: la famiglia imbuto, cinque forme con circa 592 pesi, da uno strato largo a cinque stretti, su due composizioni dell'ingresso. Passando da 1 a 5 strati la sola rete sale del 48 [44–53] % per P2 e del 63 [55–70] % per P3; P1 e P1.5 del 14–17 %.

Che cosa vuol dire per chi progetta il modello. Con le pipeline specializzate (P1, P1.5) la forma della rete si sceglie solo per l'accuratezza. Con quelle generiche (P2, P3), a parità di pesi, conviene una rete larga e poco profonda.

Sull'E-core P3 ha la stessa forma, più in alto del 16–21 %: la forma migliore non dipende dal core su cui girerà il nodo.

SLIDE 16

Leggero o saturo, la rete di P2 e P3 costa uguale



GLI ASSI SOTTO TRAFFICO · LA SOLA RETE NEURALE · P-CORE

Leggero o saturo, la rete di P2 e P3 costa uguale

P3. Media a traffico leggero / nodo saturo 1,02 [0,94–1,11] su 91 modelli; il minimo sta al 86 % della media.

P2. 1,03 [0,94–1,09]. P1 e P1.5 no: 0,77 e 0,83 (mediane), con una rete da poche decine di ns.

QUINDI Tempo di CPU a nodo saturo e T2−T1 a traffico leggero danno lo stesso costo della rete per P2 e P3, su ogni modello. Il pacchetto più veloce costa ~14 % meno della media: cache e predittori non sempre caldi.

T2−T1 (build strumentata), xdp_gen a 0,25 e 0,5 Mpps, baseline sottratta · mediane per strati, famiglia base (53 modelli) · P-core

Prima di passare alla sola rete neurale, un controllo su come la si misura. Il costo della rete si può ricavare in due modi indipendenti:

- **a nodo saturo**: il tempo di CPU per pacchetto meno quello della baseline (slide 3);
- **a traffico leggero**: con i tre orologi della slide 11, T2−T1 meno la stessa misura della baseline, con xdp_gen a 0,25 e 0,5 milioni di pacchetti al secondo.

Ogni grafico ha tre curve, mediane per numero di strati sulla famiglia base: **nodo saturo** (grigio tratteggiato), **traffico leggero, minimo** (chiaro: il pacchetto più veloce di ogni finestra, il caso migliore) e **traffico leggero, media** (scuro: il costo tipico di un pacchetto). **A sinistra P3**, **a destra P2**.

Per P3 la media a traffico leggero sta a 1,02 [0,94–1,11] volte il costo a nodo saturo, sui 91 modelli; per P2 a 1,03 [0,94–1,09]. Le due misure danno lo stesso costo, su ogni modello. Il minimo sta all'86–87 % della media.

Perché il minimo sta sotto. Fra un pacchetto e l'altro il core riceve, inoltra e serve il kernel, e una parte dello stato (dati, istruzioni, predittori dei salti) si raffredda. Il minimo è il pacchetto che trova tutto caldo; la media è il costo tipico, e coincide con quello misurato a nodo saturo.

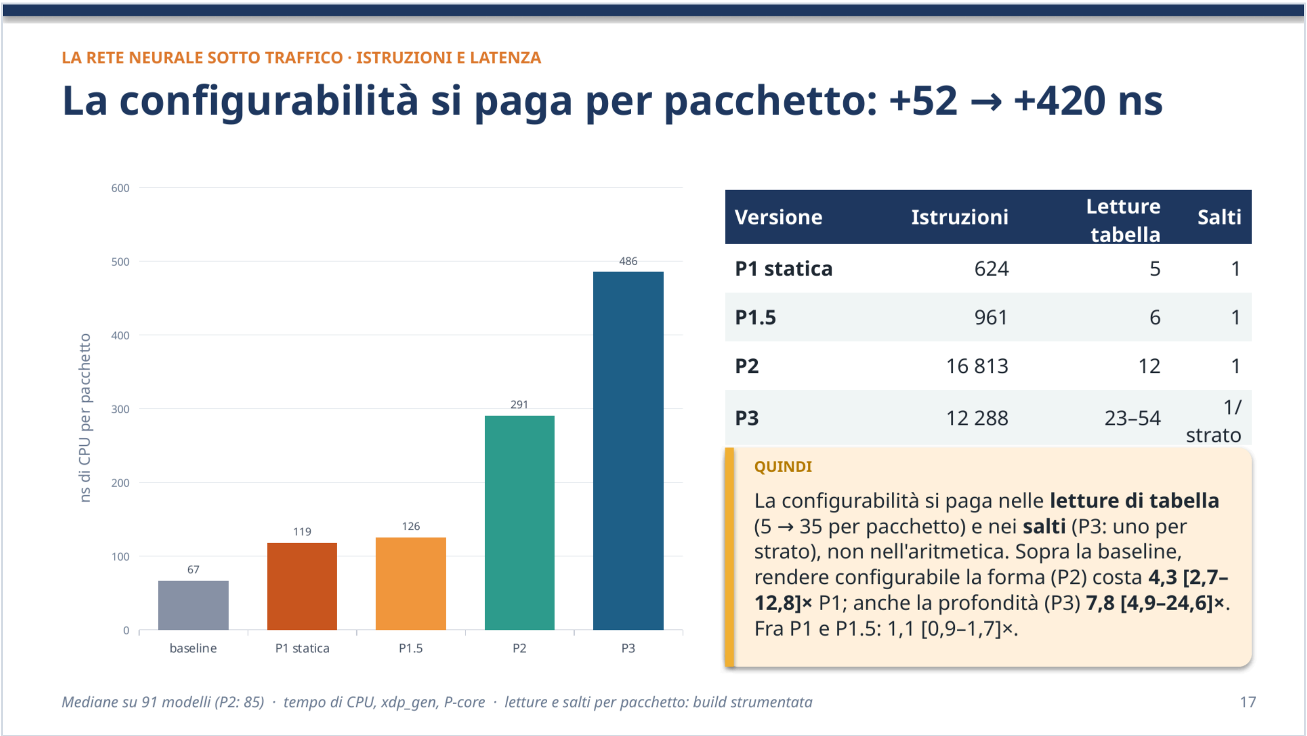
P1 e P1.5 no. Per loro il rapporto vale 0,77 [0,48–1,02] e 0,83 [0,50–1,02] sul P-core, e sull'E-core sale sopra 1 (mediana 1,27 e 1,22). La loro rete costa poche decine di ns sopra una baseline di ~67: la differenza fra due misure grandi è rumorosa. Il loro costo della rete va citato con il metodo che lo ha misurato.

> **E il programma da solo?** `bench_solo` lo misura a chiamate singole, con il pacchetto del traffico: ogni esecuzione è una chiamata al kernel, che raffredda cache e predittori come fa il traffico. Così misurata, la rete da sola costa quanto sotto traffico o poco di più (nodo saturo / da solo: 0,94 [0,77–1,10] per P2, 0,91 [0,84–0,97] per P3). Il programma in un ciclo caldo, senza nulla in mezzo, la popolazione non lo misura.

Per questo le slide 17–23 usano il **tempo di CPU per pacchetto meno la baseline**, misurato a nodo saturo con la build di produzione (senza orologi): è il costo che il nodo paga davvero.

SLIDE 17

La configurabilità si paga per pacchetto



LA RETE NEURALE SOTTO TRAFFICO · ISTRUZIONI E LATENZA

La configurabilità si paga per pacchetto: +52 → +420 ns

Versione	Istruzioni	Letture tabella	Salti
P1 statica	624	5	1
P1.5	961	6	1
P2	16 813	12	1
P3	12 288	23–54	1/ strato

QUINDI

La configurabilità si paga nelle **letture di tabella** (5 → 35 per pacchetto) e nei **salti** (P3: uno per strato), non nell'aritmetica. Sopra la baseline, rendere configurabile la forma (P2) costa **4,3 [2,7–12,8]×** P1; anche la profondità (P3) **7,8 [4,9–24,6]×**. Fra P1 e P1.5: 1,1 [0,9–1,7]×.

Mediane su 91 modelli (P2: 85) · tempo di CPU, xdp_gen, P-core · letture e salti per pacchetto: build strumentata

Da qui in poi ogni tempo è il **tempo di CPU per pacchetto** del nodo saturo sotto xdp_gen, sul P-core a 3,5 GHz (slide 3). Il costo della sola rete neurale è quel tempo meno la baseline, il programma che riceve e inoltra senza rete neurale.

Il grafico dà il tempo per pacchetto in mediana sui 91 modelli: 67 ns la baseline, 119 P1, 126 P1.5, 291 P2, 486 P3. Sopra la baseline la rete neurale costa da **+52 ns (P1) a +420 ns (P3)** in mediana.

La tabella dà tre proprietà del programma, in mediana sulla popolazione:

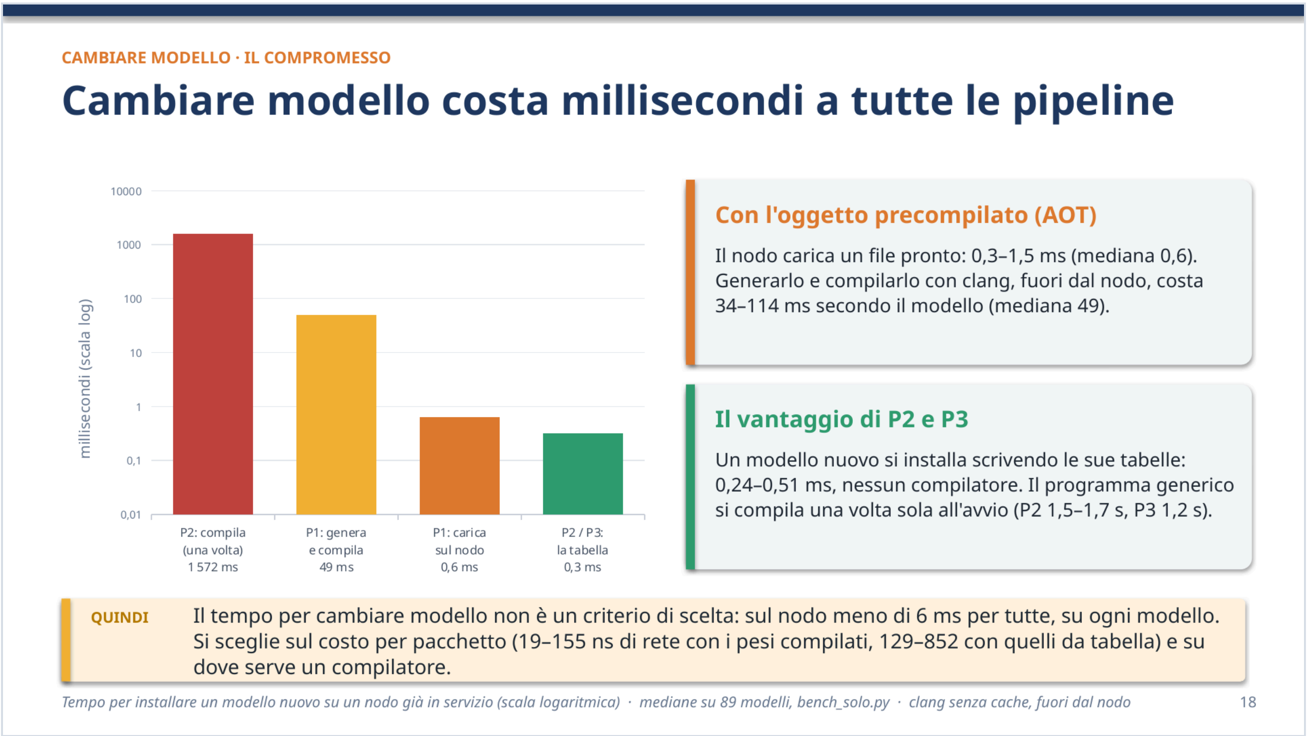
- **istruzioni**: quelle dei programmi della pipeline, contate dal kernel quando li carica;
- **letture di tabella** per pacchetto: 5 per P1, 6 per P1.5, 12 per P2, e per P3 da 23 a 54 secondo il modello, circa 6 in più a ogni strato. Le conta una build che incrementa un contatore a ogni lettura, sul pacchetto e con gli ingressi del traffico;
- **salti fra programmi** per pacchetto: uno per P1, P1.5 e P2; per P3 uno per strato, su 89 modelli su 89.

Esempio. P2 carica 16 813 istruzioni (mediana) ed esegue 3 972 istruzioni di rete per pacchetto; P3 ne carica 12 288 e ne esegue 6 969. Il programma più piccolo è il più lento: rientra nello strato generico a ogni strato (un salto), e ogni volta rilegge da tabella dove si trova (in mediana 35 letture per pacchetto contro 12).

La configurabilità si paga nelle letture di tabella e nei salti, non nell'aritmetica. Modello per modello, sopra la baseline rendere configurabile la forma (P2) costa 4,3 [2,7–12,8] volte P1; anche la profondità (P3) 7,8 [4,9–24,6] volte; P3 costa 1,9 [1,4–2,3] volte P2. Fra P1 e P1.5 quasi niente: 1,1 [0,9–1,7] volte. I rapporti più alti sono quelli dei modelli più piccoli, dove P1 costa poche decine di ns.

SLIDE 18

Cambiare modello costa millisecondi a tutte le pipeline



CAMBIARE MODELLO · IL COMPROMESSO

Cambiare modello costa millisecondi a tutte le pipeline

Con l'oggetto precompilato (AOT)

Il nodo carica un file pronto: 0,3–1,5 ms (mediana 0,6). Generarlo e compilarlo con clang, fuori dal nodo, costa 34–114 ms secondo il modello (mediana 49).

Il vantaggio di P2 e P3

Un modello nuovo si installa scrivendo le sue tabelle: 0,24–0,51 ms, nessun compilatore. Il programma generico si compila una volta sola all'avvio (P2 1,5–1,7 s, P3 1,2 s).

QUINDI Il tempo per cambiare modello non è un criterio di scelta: sul nodo meno di 6 ms per tutte, su ogni modello. Si sceglie sul costo per pacchetto (19–155 ns di rete con i pesi compilati, 129–852 con quelli da tabella) e su dove serve un compilatore.

Tempo per installare un modello nuovo su un nodo già in servizio (scala logaritmica) · mediane su 89 modelli, bench_solo.py · clang senza cache, fuori dal nodo

Questi non sono tempi per pacchetto ma tempi del **piano di controllo**: quanto costa installare un modello nuovo su un nodo già in servizio. Si misurano fuori dal traffico (`bench_solo`), su ogni modello della popolazione. Il grafico (scala logaritmica) dà le mediane:

- **P2: compilare il programma generico**, una volta sola all'avvio del nodo: 1,5–1,7 s (P3: 1,2 s);
- **P1: generare e compilare** il programma con i pesi dentro, su una macchina qualunque, fuori dal nodo: 34–114 ms secondo il modello (mediana 49). Il C si genera in meno di un millisecondo; il resto è clang, e cresce con la taglia del modello;
- **P1: caricarlo sul nodo** (verifier e JIT): 0,3–1,5 ms (mediana 0,6);
- **P2 / P3: scrivere le tabelle** con i pesi del modello nuovo: 0,24–0,51 ms.

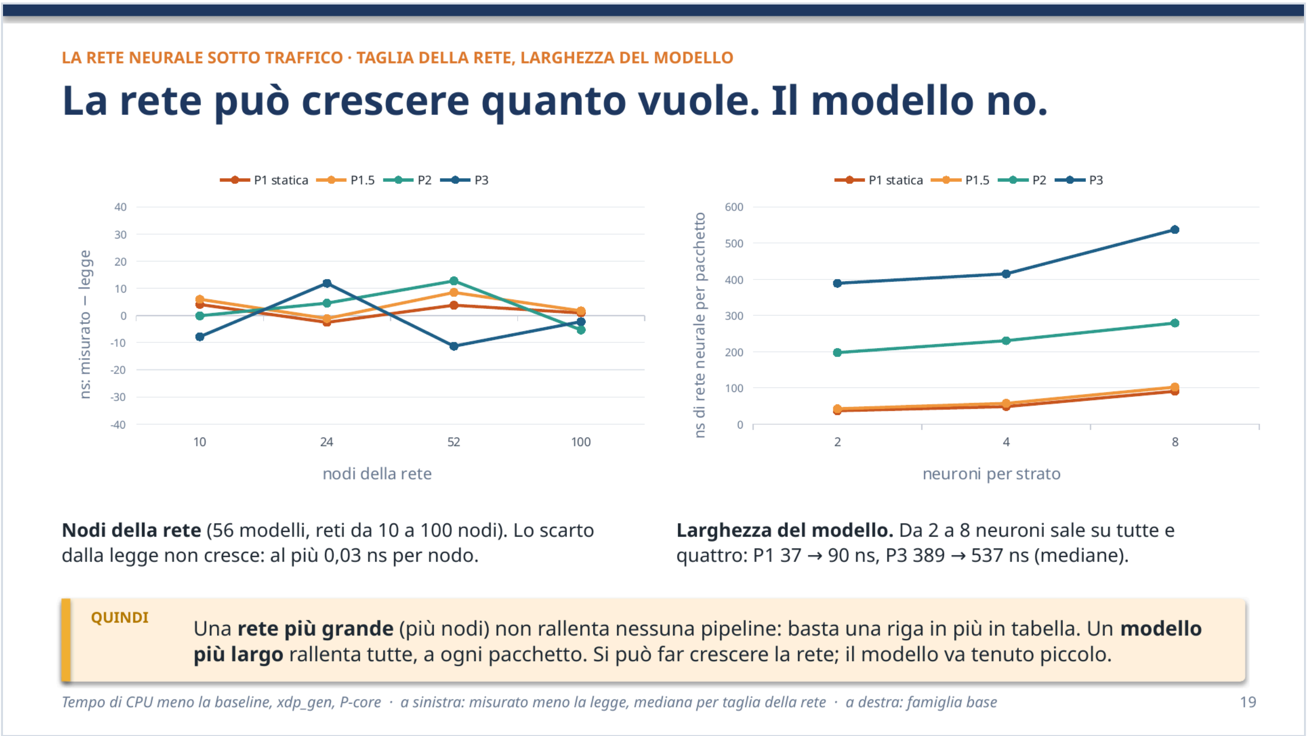
Perché P1 non ha bisogno di un compilatore sul nodo. Il programma con i pesi si compila prima (AOT, ahead of time) e al nodo arriva un file già pronto, che un piccolo caricatore apre e attacca. Il nodo non ha né clang né le librerie di sviluppo.

Il tempo per cambiare modello non è quindi un criterio di scelta: sul nodo meno di 6 ms per tutte, su ogni modello. Si sceglie sul costo per pacchetto (slide 17) e su dove serve un compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 nessuno.

Che cosa non dice. Scrivere le tabelle è misurato su un nodo che le ha già aperte: è il tempo delle scritture, non quello di un processo che parte da zero. E non dice che cosa succede ai pacchetti che passano **mentre** il modello cambia (perdite, pacchetti classificati con una tabella scritta a metà): è un test che il banco non fa ancora.

SLIDE 19

La rete può crescere quanto vuole. Il modello no.



LA RETE NEURALE SOTTO TRAFFICO · TAGLIA DELLA RETE, LARGHEZZA DEL MODELLO

La rete può crescere quanto vuole. Il modello no.

Nodi della rete (56 modelli, reti da 10 a 100 nodi). Lo scarto dalla legge non cresce: al più 0,03 ns per nodo.

Larghezza del modello. Da 2 a 8 neuroni sale su tutte e quattro: P1 37 → 90 ns, P3 389 → 537 ns (mediane).

QUINDI Una **rete più grande** (più nodi) non rallenta nessuna pipeline: basta una riga in più in tabella. Un **modello più largo** rallenta tutte, a ogni pacchetto. Si può far crescere la rete; il modello va tenuto piccolo.

Tempo di CPU meno la baseline, xdp_gen, P-core · a sinistra: misurato meno la legge, mediana per taglia della rete · a destra: famiglia base

A sinistra la taglia della rete. Nella popolazione 56 modelli hanno fra gli ingressi l'identità del nodo, una one-hot con una colonna per ogni nodo della rete: 10, 24, 52 o 100 nodi. Il grafico dà, per ogni taglia, la mediana dello **scarto dalla legge**: il costo misurato meno quello previsto dalle moltiplicazioni eseguite e dagli strati (slide 22). Se la taglia della rete costasse qualcosa, lo scarto crescerebbe con i nodi. Non cresce: sta entro ±13 ns a ogni taglia, e la pendenza è al più 0,03 ns per nodo su tutte e quattro le pipeline.

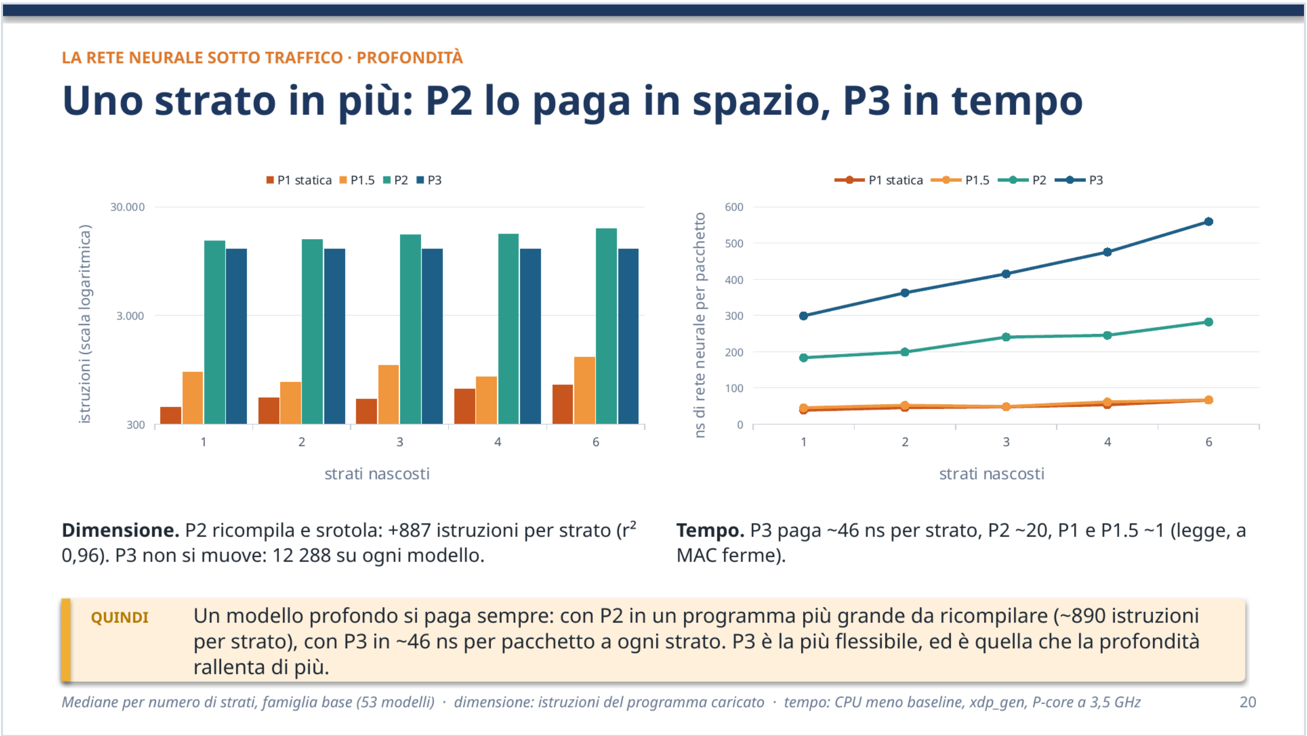
Il motivo: l'identità del nodo attiva una sola colonna di pesi, qualunque sia la larghezza della one-hot. P1 statica la conosce già quando viene compilata, le altre la leggono da tabella; nessuna moltiplica le colonne spente.

A destra la larghezza del modello: mediana per neuroni per strato sulla famiglia base. Sale su tutte e quattro: P1 37 → 90 ns, P3 389 → 537 ns di rete neurale. Un neurone in più è lavoro vero a ogni pacchetto.

Una rete più grande (più nodi) non rallenta nessuna pipeline: basta una riga in più in tabella. Un modello più largo rallenta tutte. Si può far crescere la rete; il modello va tenuto piccolo.

SLIDE 20

Uno strato in più: P2 lo paga in spazio, P3 in tempo



LA RETE NEURALE SOTTO TRAFFICO · PROFONDITÀ

Uno strato in più: P2 lo paga in spazio, P3 in tempo

Dimensione. P2 ricompila e srotola: +887 istruzioni per strato (r^2 0,96). P3 non si muove: 12 288 su ogni modello.

Tempo. P3 paga ~46 ns per strato, P2 ~20, P1 e P1.5 ~1 (legge, a MAC ferme).

QUINDI Un modello profondo si paga sempre: con P2 in un programma più grande da ricompilare (~890 istruzioni per strato), con P3 in ~46 ns per pacchetto a ogni strato. P3 è la più flessibile, ed è quella che la profondità rallenta di più.

Mediane per numero di strati, famiglia base (53 modelli) · dimensione: istruzioni del programma caricato · tempo: CPU meno baseline, xdp_gen, P-core a 3,5 GHz

Mediane per numero di strati nascosti sulla famiglia base, e leggi su tutti i modelli densi.

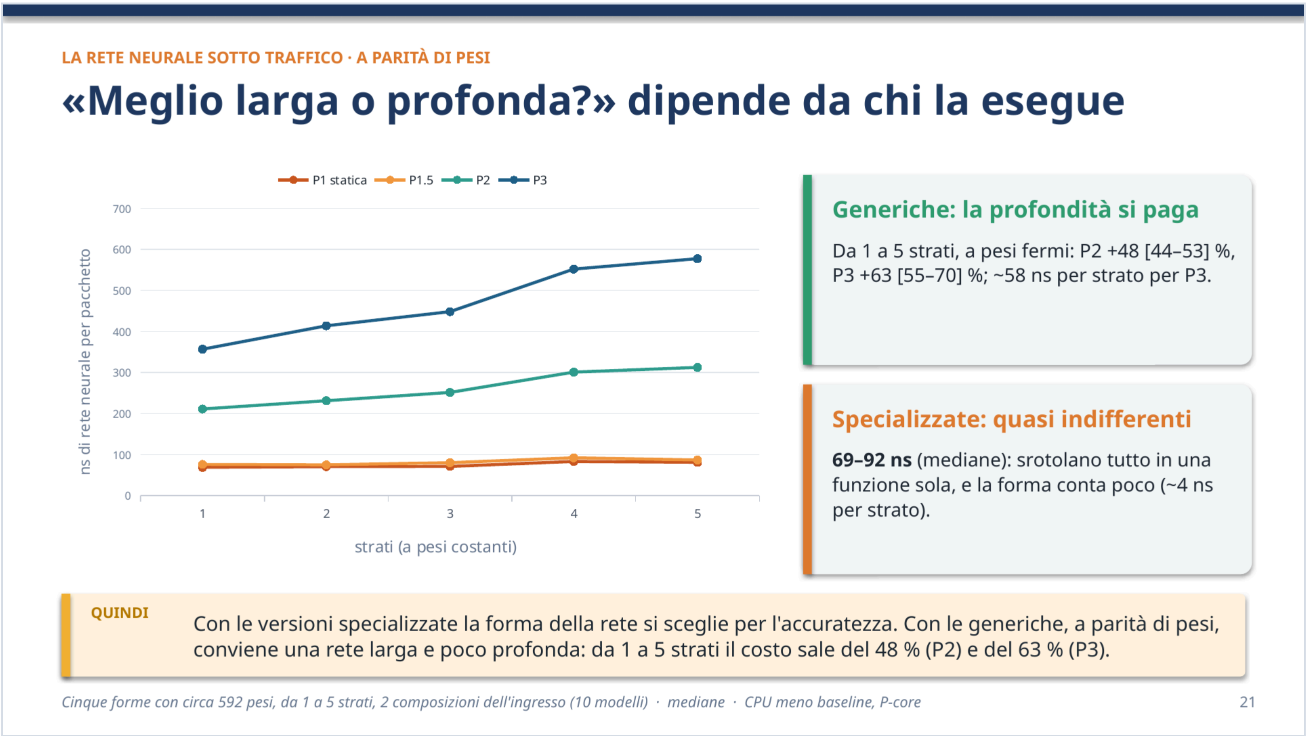
A sinistra la dimensione (scala logaritmica): le istruzioni del programma caricato. P2 ricompila e srotola per ogni profondità: la legge gli dà +887 istruzioni per strato (r^2 0,96), da 14 604 a 18 937. P3 non si muove: 12 288 istruzioni su ogni modello, perché esegue sempre lo stesso strato generico. P1 e P1.5 crescono con le moltiplicazioni, non con gli strati.

A destra il tempo sotto traffico: a moltiplicazioni ferme P3 paga circa 46 ns per strato, perché ogni strato è un salto fra programmi e un giro di letture di tabella (le ~900 istruzioni della slide 14). P2 paga circa 20 ns per strato, P1 e P1.5 circa 1.

Un modello profondo si paga sempre: con P2 in un programma più grande da ricompilare, con P3 in tempo a ogni pacchetto. P3 è la più flessibile, ed è quella che la profondità rallenta di più.

SLIDE 21

«Meglio larga o profonda?» dipende da chi la esegue



LA RETE NEURALE SOTTO TRAFFICO · A PARITÀ DI PESI

«Meglio larga o profonda?» dipende da chi la esegue

Generiche: la profondità si paga

Da 1 a 5 strati, a pesi fermi: P2 +48 [44–53] %, P3 +63 [55–70] %; ~58 ns per strato per P3.

Specializzate: quasi indifferenti

69–92 ns (mediane): srotolano tutto in una funzione sola, e la forma conta poco (~4 ns per strato).

QUINDI Con le versioni specializzate la forma della rete si sceglie per l'accuratezza. Con le generiche, a parità di pesi, conviene una rete larga e poco profonda: da 1 a 5 strati il costo sale del 48 % (P2) e del 63 % (P3).

Cinque forme con circa 592 pesi, da 1 a 5 strati, 2 composizioni dell'ingresso (10 modelli) · mediane · CPU meno baseline, P-core

La famiglia imbuto: cinque forme con circa 592 pesi, uno strato largo da 8 neuroni oppure fino a cinque strati più stretti, su due composizioni dell'ingresso (10 modelli). Lo stesso numero di pesi, disposto in modo diverso. Il grafico dà la mediana delle due composizioni.

- **Generiche (P2, P3): la profondità si paga.** Da 1 a 5 strati P2 sale del 48 [44–53] %, P3 del 63 [55–70] %; per P3 sono circa 58 ns per strato.
- **Specializzate (P1, P1.5): quasi indifferenti.** 69–92 ns di rete in mediana, circa 4 ns per strato: il compilatore srotola tutto in una funzione sola, e la forma conta poco.

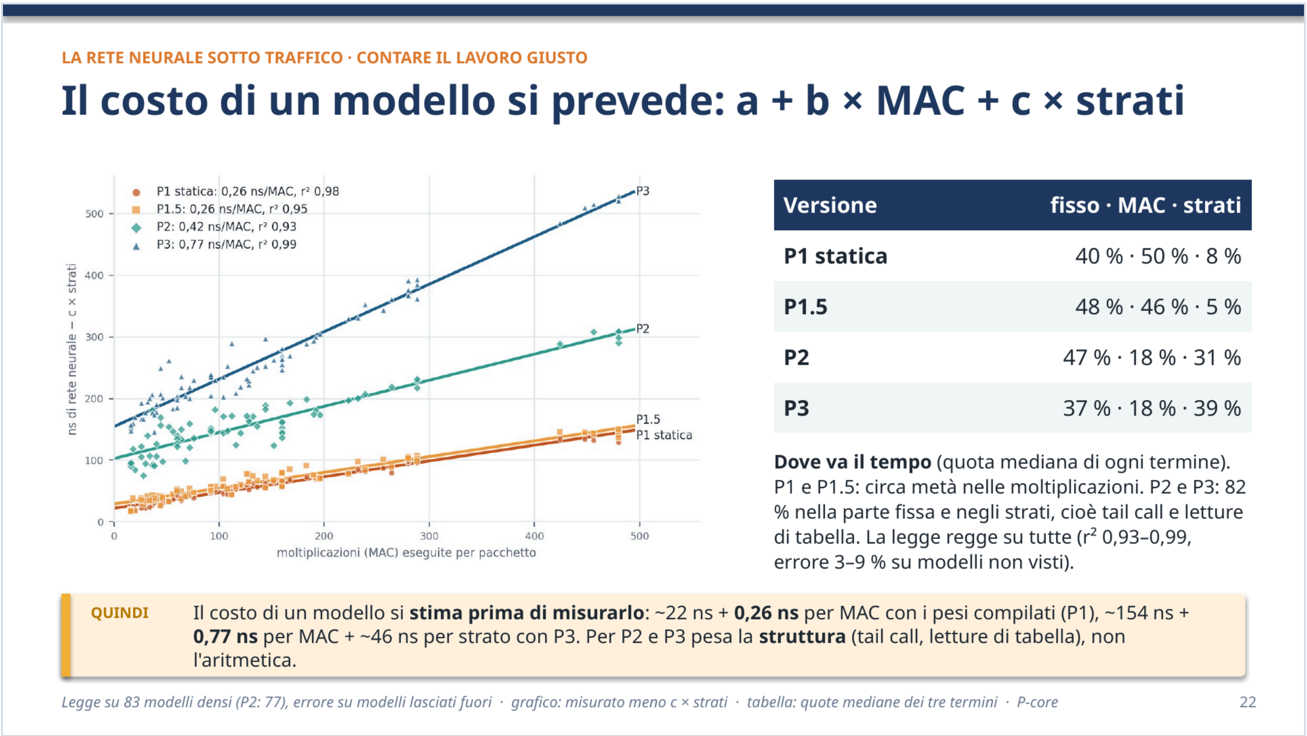
Esempio. Lo stesso budget di ~592 pesi in un solo strato costa a P3 357 ns di rete (mediana), in cinque strati 577: la scelta della forma vale più di 200 ns per pacchetto con P3, una decina con P1.

Qui lo strato costa a P3 58 ns invece dei 46 della legge: a pesi fermi, togliere neuroni da uno strato largo per farne strati stretti cambia anche le moltiplicazioni eseguite, e la legge le conta a parte.

Con le versioni specializzate la forma della rete si sceglie per l'accuratezza. Con le generiche, a parità di pesi, conviene una rete larga e poco profonda.

SLIDE 22

Il costo di un modello si prevede


P1 statica: 0,26 ns/MAC, r^2 0,98
P1.5: 0,26 ns/MAC, r^2 0,95
P2: 0,42 ns/MAC, r^2 0,93
P3: 0,77 ns/MAC, r^2 0,99
ns di rete neurale − c × strati
0
100
200
300
400
500
moltiplicazioni (MAC) eseguite per pacchetto
0
100
200
300
400
500
P3
P2
P1.5
P1 statica

LA RETE NEURALE SOTTO TRAFFICO · CONTARE IL LAVORO GIUSTO

Il costo di un modello si prevede: a + b × MAC + c × strati

Versione	fisso · MAC · strati
P1 statica	40 % · 50 % · 8 %
P1.5	48 % · 46 % · 5 %
P2	47 % · 18 % · 31 %
P3	37 % · 18 % · 39 %

Dove va il tempo (quota mediana di ogni termine). P1 e P1.5: circa metà nelle moltiplicazioni. P2 e P3: 82 % nella parte fissa e negli strati, cioè tail call e letture di tabella. La legge regge su tutte (r^2 0,93–0,99, errore 3–9 % su modelli non visti).

QUINDI Il costo di un modello si **stima prima di misurarlo**: ~22 ns + **0,26 ns** per MAC con i pesi compilati (P1), ~154 ns + **0,77 ns** per MAC + ~46 ns per strato con P3. Per P2 e P3 pesa la **struttura** (tail call, letture di tabella), non l'aritmetica.

Legge su 83 modelli densi (P2: 77), errore su modelli lasciati fuori · grafico: misurato meno c × strati · tabella: quote mediane dei tre termini · P-core

Si può prevedere quanto costerà un modello prima di misurarlo? La legge stimata sugli 83 modelli densi (77 per P2) è

costo della rete = a + b × moltiplicazioni eseguite + c × strati

con questi coefficienti sul P-core:

- **P1**: 22 ns + 0,26 ns per moltiplicazione + 1 ns per strato (r^2 0,98);
- **P1.5**: 29 ns + 0,26 ns + 1 ns (r^2 0,95);
- **P2**: 103 ns + 0,42 ns + 20 ns (r^2 0,93);
- **P3**: 154 ns + 0,77 ns + 46 ns (r^2 0,99).

Il trucco è contare le moltiplicazioni **davvero eseguite**. Un ingresso one-hot occupa molte colonne nella matrice dei pesi ma ne attiva una sola: contato dalla forma del modello sembra costoso, nel programma no. Contando le moltiplicazioni dalla forma la legge spiega meno (r^2 0,58–0,79); contando quelle eseguite spiega il costo di tutte e quattro le pipeline.

Il grafico mostra ogni modello denso: sull'asse x le moltiplicazioni eseguite, sull'asse y il costo della rete meno c × strati, cioè la parte che la legge attribuisce alle moltiplicazioni. I punti cadono sulle rette.

Dove va il tempo. La tabella dà, per ogni pipeline, la quota mediana di ciascun termine sul costo previsto dei modelli densi (fisso · moltiplicazioni · strati):

- **P1**: 40 · 50 · 8 %; **P1.5**: 48 · 46 · 5 %. Con i pesi nel codice circa metà del tempo è aritmetica;
- **P2**: 47 · 18 · 31 %; **P3**: 37 · 18 · 39 %. Con i pesi in tabella circa l'82 % del tempo è **struttura**: il termine fisso (dispatcher, lettura del descrittore del modello, costruzione degli ingressi da tabella) e quello per strato (la tail call allo strato generico e la rilettura del suo contesto, le ~900 istruzioni della slide 14). Le moltiplicazioni arrivano al 44–46 % solo sui modelli più grandi.

Anche il termine per moltiplicazione contiene le tabelle: P3 paga 0,77 ns per MAC contro 0,26 di P1 perché legge ogni peso dalla memoria di una mappa, in un ciclo non srotolato, mentre in P1 il peso è una costante nel codice. La legge resta lineare proprio perché ogni tail call e ogni lettura di tabella costa quasi sempre lo stesso, e il loro numero dipende solo dalla forma.

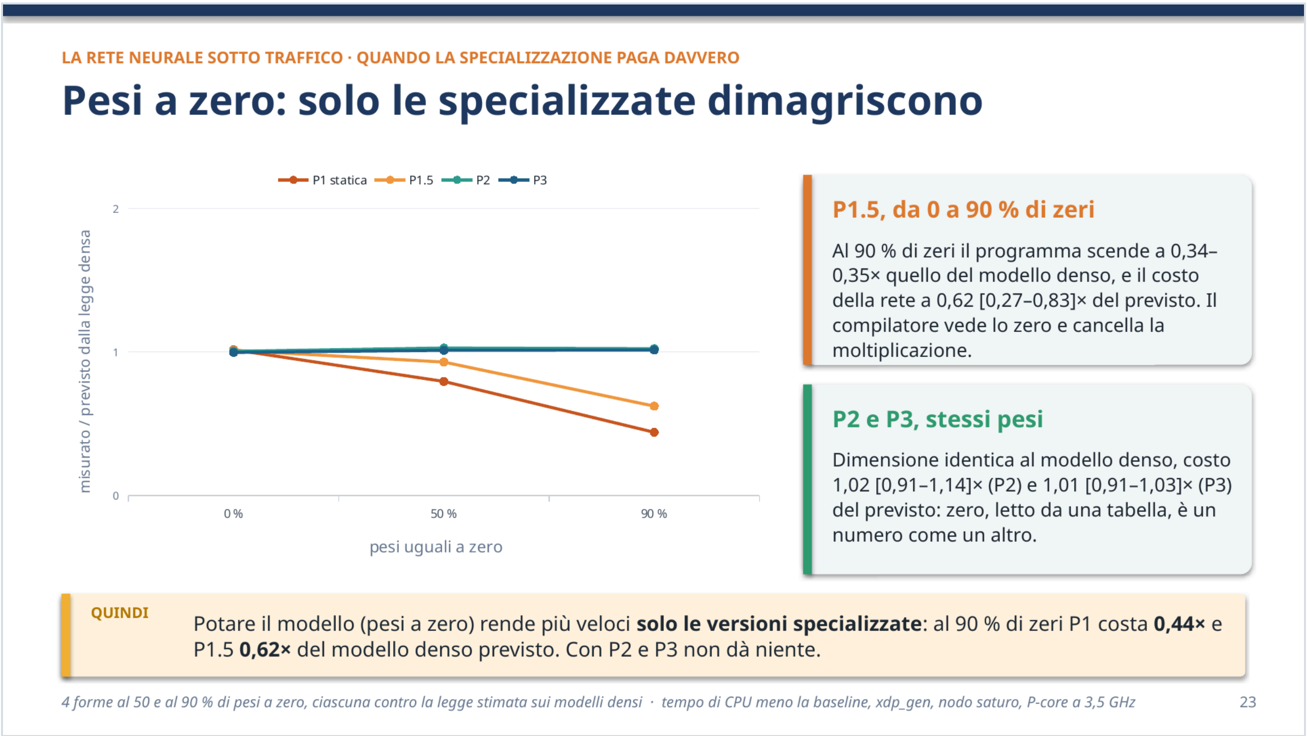
> **Nessun passaggio in user space.** Le mappe si leggono dentro il kernel (`bpf_map_lookup_elem`; per le mappe array il verifier sostituisce di solito la chiamata con un calcolo di indirizzo). Lo user space scrive le mappe solo quando si installa un modello, mai mentre passano i pacchetti. Le due cose si contano: su ogni modello P3 fa 35 [23–54] letture di tabella per pacchetto, circa 6 in più per strato, e un salto per strato su 89 modelli su 89. Quanto dei 46 ns per strato sia il salto e quanto le letture, invece, la campagna non lo separa.

> **Su modelli che non ha visto.** La legge si stima lasciando fuori un modello alla volta e prevedendo quello: l'errore mediano è del 3 % per P3, del 6 % per P1 e P2, del 9 % per P1.5. Il costo di un modello nuovo si stima quindi prima di misurarlo, con qualche punto percentuale di errore.

Anche i pesi estratti contano poco: sei forme ripetute con quattro semi diversi (uno con pesi a distribuzione normale) danno un costo che varia al più del 5–6 % per P1, P1.5 e P2, del 2 % per P3.

SLIDE 23

Pesi a zero: solo le specializzate dimagriscono



LA RETE NEURALE SOTTO TRAFFICO · QUANDO LA SPECIALIZZAZIONE PAGA DAVVERO

Pesi a zero: solo le specializzate dimagriscono

P1.5, da 0 a 90 % di zeri

Al 90 % di zeri il programma scende a 0,34–0,35× quello del modello denso, e il costo della rete a 0,62 [0,27–0,83]× del previsto. Il compilatore vede lo zero e cancella la moltiplicazione.

P2 e P3, stessi pesi

Dimensione identica al modello denso, costo 1,02 [0,91–1,14]× (P2) e 1,01 [0,91–1,03]× (P3) del previsto: zero, letto da una tabella, è un numero come un altro.

QUINDI Potare il modello (pesi a zero) rende più veloci **solo le versioni specializzate**: al 90 % di zeri P1 costa **0,44×** e P1.5 **0,62×** del modello denso previsto. Con P2 e P3 non dà niente.

4 forme al 50 e al 90 % di pesi a zero, ciascuna contro la legge stimata sui modelli densi · tempo di CPU meno la baseline, xdp_gen, nodo saturo, P-core a 3,5 GHz

23

Le quattro forme della famiglia sparsità, ciascuna con il 50 e il 90 % dei pesi esattamente a zero. Il grafico dà, per ogni livello, il costo misurato della rete diviso quello che la legge della slide 22, stimata sui modelli densi, prevede per la stessa forma.

- **P1 e P1.5** scendono sotto la legge: al 90 % di zeri P1 costa 0,44 [0,31–0,63] volte il previsto, P1.5 0,62 [0,27–0,83]. Il programma di P1.5 scende a 0,34–0,35 volte quello del modello denso con la stessa forma (al 50 %: 0,78): il compilatore vede che il peso è zero e cancella la moltiplicazione.
- **P2 e P3** stanno sulla legge: 1,02 [0,91–1,14] e 1,01 [0,91–1,03] al 90 %, con un programma identico a quello del modello denso. Uno zero letto da una tabella è un numero come un altro, e la moltiplicazione si fa comunque.

È l'unico esperimento in cui dimensione e tempo scendono insieme. Potare il modello (mettere pesi a zero) rende più veloci solo le versioni specializzate; con P2 e P3 la potatura non dà niente in velocità, e va giustificata solo dall'accuratezza.